汽车电子与控制
Automotive Electronics and Control Systems (AECS)

李建秋、徐梁飞
清华大学车辆与运载学院
汽车安全与节能国家重点实验室
2020.9

第二章：汽车电控系统数字核心（单片机）
Digital Core of AECS (Microcontroller)

§目录 / Index

1 TC系列MCU结构与编程

2 TC系列MCU指令系统

3 TC系列MCU的CPU与外围电路

4 TC系列MCU的周边模块

2 TC系列MCU指令系统

- (1) 基本概念
- (2) 典型指令集
  - Integere Arithmetic
  - Packed Arithmetic
  - DSP Arithmetic
  - Compare Instructions
  - Bit Operations
  - Address Arithmetic
  - Address Comparison
  - Branch Instructions
  - Load and Store Instructions
  - Context Related Instructions
  - System Instructions
  - Float Point Co-Processor

（1）基本概念

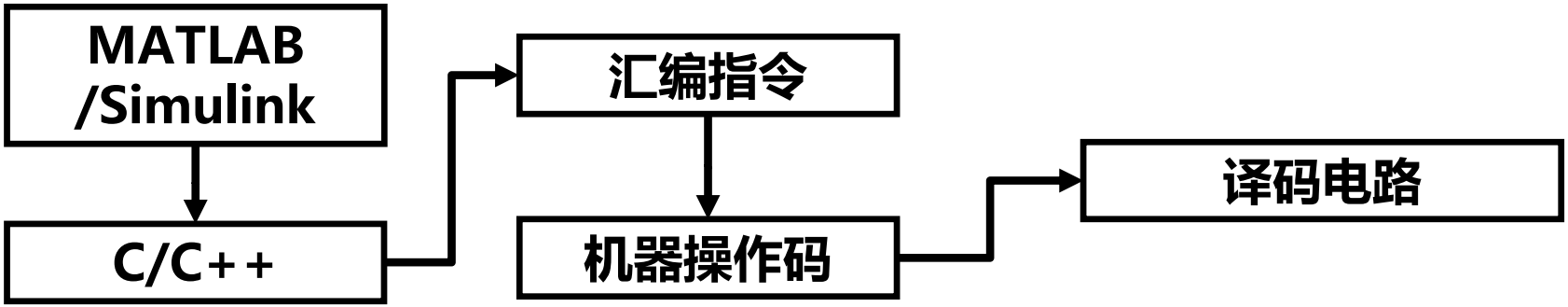
指令码 instruction

Examples

```
mov.a    a3, d1
```

操作数operator

```
mov.a    a4, d2
mov.a    a4, 7
```



Instruction Syntax（指令语法）

◆指令基本格式（三个部分）

An instruction mnemonic is composed of up to three basic parts.

- A base operation
  - Specifies the instructions basic operation. For example: ADD for addition, J for jump and LD for memory load. Some instructions such as OR.EQ, have more than one base operation, separated by a period (.)
- An operation modifier
  - Specifies the operation more precisely. For example: ADDI for addition using an immediate value, or JL for a jump that includes a link. More than one operation modifier may be used for some instructions (ADDIH for example).
- An operand (data type) modifier.
  - Gives the data type of the source operands. For example: ADD.B for byte addition, JZ.A for a jump using an address register and LD.H for a half-word load. The data type modifier is separated by a period (.).

Using the ADDS.U instruction as an example:

- 'ADD' is the base operation.
- 'S' is an operation modifier specifying that the result is saturated.
- 'U' is a data type modifier specifying that the operands are unsigned.

Instruction Syntax

◆操作修饰符

Table 1-3 Operation Modifiers

Operation Modifier	Name	Description	Example
C	Carry	Use and update PSW carry bit	ADDC
I	Immediate	Large immediate	ADDI
H	High Word	Immediate value put in most-significant bits	ADDIH
S	Saturation	Saturate result	ADDS
X	Carry out	Update PSW carry bit	ADDX
EQ	Equal	Comparison equal	JEQ
GE	Greater than	Comparison greater than or equal	JGE
L	Link	Record link (jump subroutine)	JL
A	Absolute	Absolute (jump)	JLA
I	Indirect	Register indirect (jump)	JLI
LT	Less than	Comparison less than	JLT
NE	Not equal	Comparison not equal	JNE
D	Decrement	Decrement counter	JNED
I	Increment	Increment counter	JNEI
Z	Zero	Use zero immediate	JNZ
M	Multi-precision	Multi-precision result (>32-bit) in Q format	MULM
R	Round	Round result	MULR
N	Not	Logical NOT	SELN

● Instruction Syntax

◆Operand操作数

Table 1-1 Operand Definitions

Operand	Definition
D[n]	Data register n
A[n]	Address register n
E[n]	Extended data register n containing a 64-bit value made from an even/odd pair of registers (D[n], D[n+1]). The format is little endian. E[n][63:32] = D[n+1][31:0]; E[n][31:0] = D[n][31:0]
dispn	Displacement value of ‘n’ bits, used to form the effective address in branch instructions
constn	Constant value of ‘n’ bits, used as instruction operand
offn	Offset value of ‘n’ bits, used to form the effective address in Load and Store instructions
pos1, pos2	Used to specify the position in a bit field instruction
pos	Pos (position) is used with width to define a field
width	Specifies the width of the bit field in bit field instructions

Instruction Syntax

◆默认操作寄存器

Table 1-2 Implicit Operand Definitions

Operand	Definition
D[15]	Implicit Data register for many 16-bit instructions
A[10]	Stack Pointer (SP)
A[11]	Return Address (RA) register for CALL, JL, JLA and JLI instructions, and Return PC value on interrupts
A[15]	Implicit Address Register for many 16-bit Load/Store instructions

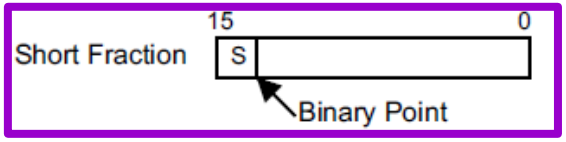
Note: In the syntax section of the instruction descriptions, the implicit registers are included as explicit operands. However they are not explicitly encoded in the instructions.

Instruction Syntax

◆数据类型说明

Table 1-4 Data Type Modifiers

Data Type Modifier	Name	Description	Example
D	Data	32-bit data	MOV.D
D	Double-word	64-bit data/address	LD.D
W	Word	32-bit (word) data	EQ.W
A	Address	32-bit address	ADD.A
Q	Q Format	16-bit signed fraction (Q format)	MADD.Q
H	Half-word	16-bit data or two packed half-words	ADD.H
B	Byte	8-bit data or four packed bytes	ADD.B
T	Bit	1-bit data	AND.T
U	Unsigned	Unsigned data type	ADDS.U



- **Instruction Syntax**
  - ◆语法格式

    ‘C-like’ Register Transfer Level (RTL) notation寄存器转换级符号

Table 1-8 RTL Syntax

Syntax	Definition
bpos3	Bit position
const’n’	Constant value of ‘n’bits, used as instruction operand
disp’n’	Displacement value of ‘n’ bits, used to form the effective address in branch instructions
(expression)[p]	A single bit, wiht ordinal index ‘p’ in the bit field ‘expression’
n’bx	Constant bit string, where ‘n’ is the number of bits in the constant and ‘x’ is the constant in binary. For example; 2’b11
n’hx	Constant bit string, where ‘n’ is the number of bits in the constant and ‘x’ is the constant in hexadecimal. For example, 16’hFFFF
off’n’	Offset value of ‘n’ bits, used to form the effective address in Load and Store instructions

Instruction Syntax

‘C-like’ Register Transfer Level (RTL) notation

pos	Single bit position
signed	A value that can be positive, negative or zero
ssov	Saturation on signed overflow
suov	Saturation on unsigned overflow
unsigned	A value that can be positive or zero
{x,y}	A bit string where ‘x’ and ‘y’ are expressions representing a bit or bit field. Any number of expressions can be concatenated. For example, {x,y,z}
A[n]	Address register ‘n’
CR	Core Register
D[n]	Data register ‘n’
EA	Effective Address
E[n]	Data register containing a 64-bit value, constructed by pairing two data registers. The least-significant bit is in the even register D[n], and the most significant bit is in the odd register D[n+1]
M(EA, data_size)	Memory locations beginning at the specified byte location EA, and extending to EA + data_size - 1. data_size = byte, half-word, word, , 16-word
<mode>	And addressing mode
P[n]	Address register containing a 64-bit value, constructed by pairing two address registers. The least-significant bit is in the even register A[n], and the most significant bit is in the odd register A[n+1]
PC	The address of the instruction in memory
[x:y]	Bits y, y+1, ..., x where x>y; For example D[a][x:y], if x=y then this is a single bit range which is also denoted by [x], as in D[a][x]. For cases where x<y, this denotes an empty range.

● Instruction Syntax

'C-like' Register Transfer Level (RTL) notation

Table 1-8 RTL Syntax

Syntax	Definition
TRUE	Boolean true. Equivalent to integer 1
FALSE	Boolean false. Equivalent to integer 0
AND	Logical AND. Returns a boolean result
OR	Logical OR. Returns a boolean result
XOR	Logical XOR. Returns a boolean result
!	Logical NOT. Returns a boolean result
^	Bitwise XOR
&	Bitwise AND
|	Bitwise OR
~	Bitwise NOT
<	Less than. Returns a boolean result
>	Greater than. Returns a boolean result
<=	Less than or equal to. Returns a boolean result
>=	Greater than or equal to. Returns a boolean result
>>	Right shift. High order bits shifted in are 0's
<<	Left shift. Low order bits shifted in are 0's
+	Add
-	Subtract
*	Multiply
/	Divide
%	Modulo
=	Equal to (assignment)
==	Is equal to (comparison). Returns a boolean result
!=	Not equal to. Returns a boolean result
≈	Approximately equal to
||	Parallel operation
? :	Conditional expression (Ternary operator)
∞	Infinity
//	Comment

- **Instruction Syntax**

  Register Transfer Level (RTL) Functions寄存器转换级函数

  CPU的Cache缓存操作指令

Table 1-9 RTL Functions

Function	Definition
abs(x)	abs(x) returns ((x <0) ? (0 - x) : x);
cache_address_ivld(EA)	Defined in 'Cache RTL Functions', which follows
cache_address_wb(EA)	Defined in 'Cache RTL Functions', which follows
cache_address_wi(EA)	Defined in 'Cache RTL Functions', which follows
cache_index_ivld	Defined in 'Cache RTL Functions', which follows
cache_index_wb	Defined in 'Cache RTL Functions', which follows
cache_index_wi	Defined in 'Cache RTL Functions', which follows
cache_index_wb(EA)	Defined in the 'Cache RTL Functions' section, which follows. (TriCore1.6.1 TriCoreS)
cache_index_wi(EA)	Defined in the 'Cache RTL Functions' section, which follows. (TriCore1.6.1 TriCoreS)

- **Instruction Syntax**

Register Transfer Level (RTL) Functions

CPU的Cache操作指令

Table 1-10 Cache Functions

Function	Definition
cache_address_ivld(EA)	if (CACHE[EA].valid==1) then CACHE [EA].valid=0;

cache_address_wb(EA)	if ((CACHE[EA].valid==1) AND (CACHE[EA].modified==1)) then {
	pa = CACHE[EA].physical_address;
	M[pa,cacheline] = CACHE[EA].data;
	CACHE[EA].modified = 0;
	}

cache_address_wi(EA)	if (CACHE[EA].valid==1) then {
	if (CACHE[EA].modified==1) then {
	pa = CACHE[EA].physical_address;
	M[pa,cacheline] = CACHE[EA].data;
	}
	CACHE[EA].modified = 0;
	CACHE[EA].valid = 0;
	}

cache_index_ivld	if (CACHE[index,elem].valid = = 1) then CACHE[index,elem].valid = 0;

cache_index_wb	if ((CACHE[index,elem].valid==1) AND (CACHE[index,elem].modified==1))
	then {
	pa = CACHE[index,elem].physical_address;
	M[pa,cacheline] = CACHE[index,elem].data;
	CACHE[index,elem].modified = 0;
	}

- **Instruction Syntax**

  Register Transfer Level (RTL) Functions

  Floating Point Operation Syntax（浮点运算语法）

ADD_NAN	$7FC00001_H$
MUL_NAN	$7FC00002_H$
SQRT_NAN	$7FC00004_H$
DIV_NAN	$7FC00008_H$
POS_INFINITY	$7F800000_H$
NEG_INFINITY	$FF800000_H$

Instruction Syntax

Register Transfer Level (RTL) Functions

is_s_nan(x)	Takes the IEEE-754-1985 32-bit single precision floating point format value 'x' and returns the boolean result of the expression: (x[30:23] == 8'b11111111) AND (x[22] == 1'b0) AND (x[21:0] != 0);
is_q_nan(x)	Takes the IEEE-754-1985 32-bit single precision floating point format value 'x' and returns the boolean result of the expression: (x[30:23] == 8'b11111111) AND (x[22] == 1'b1);
is_nan(x)	Takes the IEEE-754-1985 32-bit single precision floating point format value 'x' and returns the boolean result of the expression: (is_s_nan(x) OR is_q_nan(x));
is_pos_inf(x)	Takes the IEEE-754-1985 32-bit single precision floating point format value 'x' and returns the boolean result of the expression: (x[31:0] == POS_INFINITY);
is_neg_inf(x)	Takes the IEEE-754-1985 32-bit single precision floating point format value 'x' and returns the boolean result of the expression: (x[31:0] == NEG_INFINITY);
is_inf(x)	Takes the IEEE-754-1985 32-bit single precision floating point format value 'x' and returns the boolean result of the expression: (is_neg_inf(x) OR is_pos_inf(x));
is_zero(x)	Takes the IEEE-754-1985 32-bit single precision floating point format value 'x' and returns the boolean result of the expression: (x[30:0] == 0);
is_denorm(x)	Takes the IEEE-754-1985 32-bit single precision floating point format value 'x' and returns the boolean result of the expression: (x[30:23] == 0) AND (x[22:0] != 0);

denorm_to_zero(x)	If the IEEE-754-1985 32-bit single precision floating point format value 'x' is a denormal value return the appropriately signed infinitely accurate real value 0. Otherwise return 'x' as an infinitely accurate real value; i.e.
	if((x < 0) AND (x > -2-126)) then return -0.0;
	else if((x > 0) AND (x < 2126)) then return +0.0;
	else return f_real(x);

Instruction Syntax

Register Transfer Level (RTL) Functions

Coprocessor Instructions（协处理器指令）

1.4 Coprocessor Instructions

The TriCore® instruction set architecture may be extended with implementation defined, application specific coprocessor instructions. These instructions are executed on dedicated coprocessor hardware attached to the coprocessor interface.

The coprocessors operate in a similar manner to the integer instructions, receiving operands from the general purpose data registers, returning a result to the same registers.

The architecture supports the operation of up to four concurrent coprocessors (n = 0, 1, 2, 3).

Two of these (n = 0, 1) are reserved for use by the TriCore CPU, allowing two (n = 2, 3) for use by the application hardware.

COPROCESSOR D[d], D[a], D[b] ()

31 28	27 20	19 18	17 16	15 12	11 8	7 0
c	op2	-	n	b	a	$4B_H$

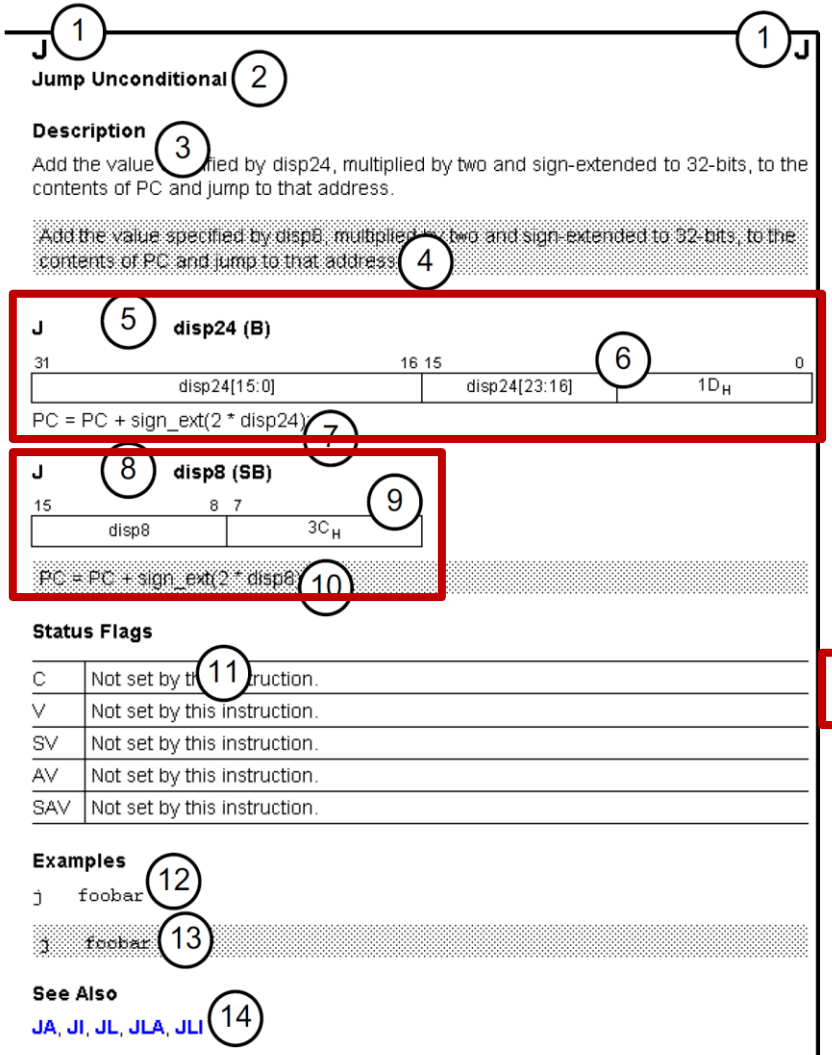
D[c] = op2[n] (D[a], D[b]);

COPROCESSOR D[c], D[d], D[a], D[b] (R)

31 28	27 24	23 20	19 18	17 16	15 12	11 8	7 0
c	d	op2	-	n	b	a	$6B_H$

D[c] = op2[n] (D[d], D[a], D[b]);

（2）典型指令集：如何阅读文档



J (1) (1) J

Jump Unconditional (2)

Description (3)

Add the value specified by disp24, multiplied by two and sign-extended to 32-bits, to the contents of PC and jump to that address.

Add the value specified by disp8, multiplied by two and sign-extended to 32-bits, to the contents of PC and jump to that address. (4)

J (5) **disp24 (B)**

31	16 15	0
disp24[15:0]	disp24[23:16]	$1D_H$

(6)

PC = PC + sign_ext(2 * disp24) (7)

J (8) **disp8 (SB)**

15 8	7
disp8	$3C_H$

(9)

PC = PC + sign_ext(2 * disp8) (10)

Status Flags

C	Not set by th (11) ruction.
V	Not set by this instruction.
SV	Not set by this instruction.
AV	Not set by this instruction.
SAV	Not set by this instruction.

Examples

```
j    foobar (12)
```

```
j    foobar (13)
```

See Also

JA, JI, JL, JLA, JLI (14)

TC1066

Key:

1) Instruction Mnemonic
2) Instruction Longname
3) Description (32-bit)
4) Description (16-bit)
5) Syntax (32-bit), followed by Instruction format in parentheses
6) Opcodes (32-bit)
7) Operation in RTL format (32-bit)
8) Syntax (16-bit)
9) Opcodes (16-bit)
10) Operation (RTL) (16-bit)
11) Status Flags (User Status Bits)
12) One or more Instruction Examples (32-bit)
13) Instruction Examples (16-bit)
14) Links to related instructions

PSW寄存器高8位信息USB

指令代码及含义

汇编语言、机器操作码和C语言RTL格式三种表达方式

实际代码编写，比如寄存器A[10]应该写成a10

（2）典型指令集：如何阅读文档

Table 1-6 32-bit Opcode Formats

	31:30	29:28	27:26	25:24	23:22	21:20	21:20	19:18	17:16	15:14	13:12	11:10	9-8	7-6	5-0
ABS	off18 [9:6]		op2	off18 [13:10]		off18[5:0]				off18 [17:14]		s1/d		op1	
ABSB	off18 [9:6]		op2	off18 [13:10]		off18[5:0]				off18 [17:14]		b	bpos3	op1	
B	disp24[15:0]									disp24[23:16]				op1	
BIT	d		pos2		op2	pos1				s2		s1		op1	
BO	off10[9:6]		op2			off10[5:0]				s2		s1/d		op1	
BOL	off16[9:6]		off16[15:10]			off16[5:0]				s2		s1/d		op1	
BRC	op2	disp15								const4		s1		op1	
BRN	op2	disp15								n[3:0]		s1		n[4] op1	
BRR	op2	disp15								s2		s1		op1	
RC	d		op2			const9						s1		op1	
RCPW	d		pos		op2	width				const4		s1		op1	
RCR	d		s3		op2	const9						s1		op1	
RCRR	d		s3		op2	-				const4		s1		op1	
RCRW	d		s3		op2	width				const4		s1		op1	
RLC	d		const16									s1		op1	
RR	d		op2				-		n	s2		s1		op1	
RR1	d		op2						n	s2		s1		op1	
RR2	d		op2							s2		s1		op1	
RRPW	d		pos		op2	width				s2		s1		op1	
RRR	d		s3		op2		-		n	s2		s1		op1	
RRR1	d		s3		op2				n	s2		s1		op1	
RRR2	d		s3		op2					s2		s1		op1	
RRRR	d		s3		op2	-				s2		s1		op1	
RRRW	d		s3		op2	width				s2		s1		op1	
SYS	-		op2			-						s1/d		op1	

机器码格式

（2）典型指令集：如何阅读文档

Table 1-5 16-bit Opcode Formats

	15-14	13-12	11-10	09-08	07-06	05-04	03-02	01-00
SB	disp8				op1			
SBC	const4		disp4		op1			
SBR	s2		disp4		op1			
SBRN	n		disp4		op1			
SC	const8				op1			
SLR	s2		d		op1			
SLRO	off4		d		op1			
SR	op2		s1/d		op1			
SRC	const4		s1/d		op1			
SRO	s2		off4		op1			
SRR	s2		s1/d		op1			
SRRS	s2		s1/d		n	op1		
SSR	s2		s1		op1			
SSRO	off4		s1		op1			

思考：在本款单片机中，一个32bit可能代表什么信息？
内存地址、寄存器、数据、机器码…

（2）典型指令集：如何阅读文档

JA
Jump Unconditional Absolute

Description

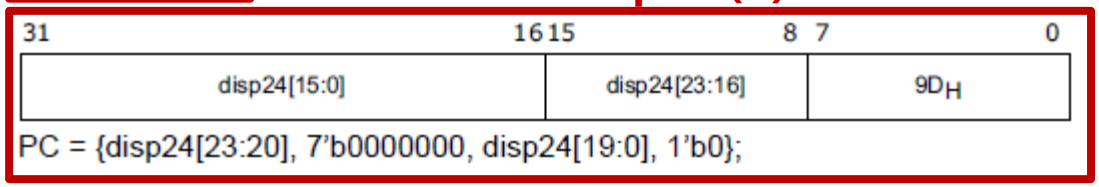
Load the value specified by disp24 into PC and jump to that address.

The value disp24 is used to form the Effective Address (EA).

JAdisp24 (B)

文档编辑失误，缺空格

JA disp24 (B)



问：如何通过disp24这个变量生成有效内存地址EA?

Status Flags

C	Not set by this instruction.
V	Not set by this instruction.
SV	Not set by this instruction.
AV	Not set by this instruction.
SAV	Not set by this instruction.

Examples

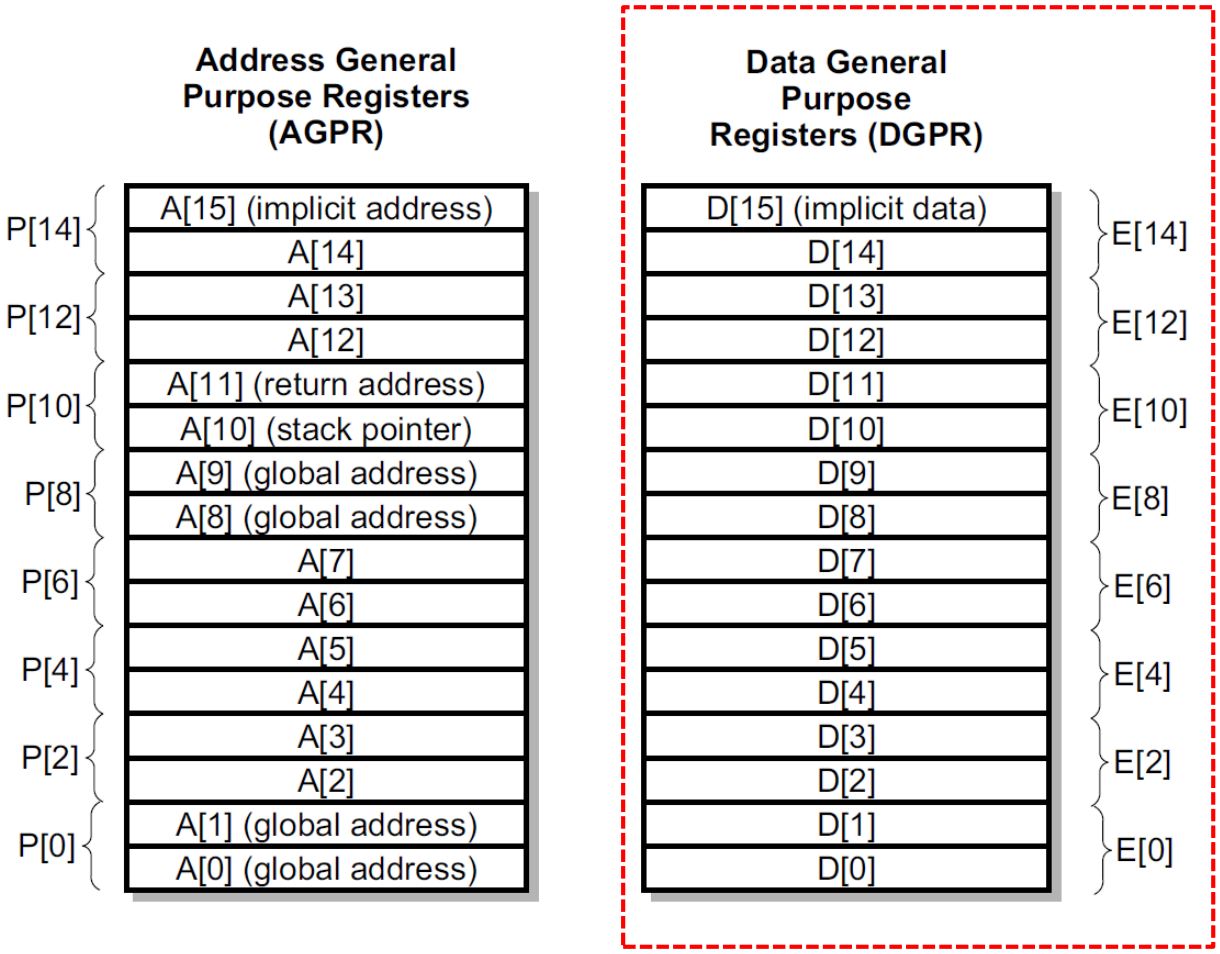
```
ja          foobar
```

注意：汇编语言、机器操作码和C语言RTL格式三种表达方式

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Move



MOV
MOV.A
MOV.AA
MOV.D
MOV.U
MOVH
MOVH.A

(2) 典型指令集

Integer Arithmetic（整数算术指令） | **Move**

MOV
Move

Description

Move the contents of either data register D[b] (instruction format RR) or const16 (instruction format RLC), to data register D[c]. The value const16 is sign-extended to 32-bits before it is moved.

The syntax is also valid with a register pair as a destination. If there is a single source operand, it is sign-extended to 64-bits. If the source is a register pair the contents of data register D[a] go into E[c][63:32] and the contents of data register D[b] into E[c][31:0].

Move the contents of either data register D[b] (instruction format SRR), const4 (instruction format SRC) or const8 (instruction format SC) to either data register D[a] (formats SRR, SRC) or D[15] (format SC). The value const4 is sign-extended before it is moved. The value const8 is zero-extended before it is moved.

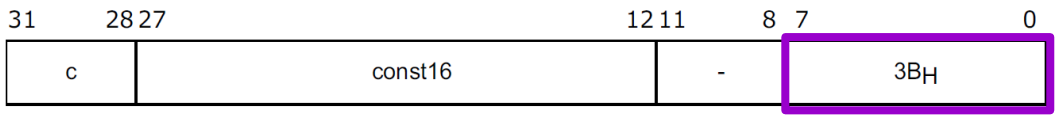
(2) 典型指令集

Integer Arithmetic（整数算术指令）

Move

MOVD[c], const16 (RLC)

MOV D[c], const16 (RLC)

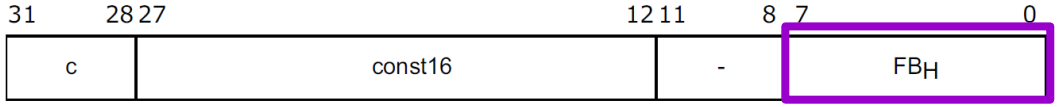


D[c] = sign_ext(const16);

机器操作码 opcode

MOVE[c], const16 (RLC)

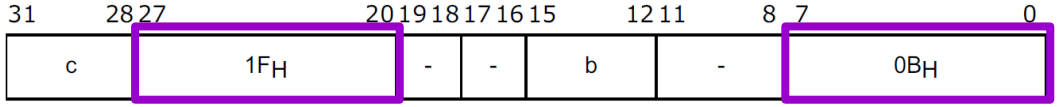
MOV E[c], const16 (RLC)



E[c] = sign_ext(const16);

MOVD[c], D[b] (RR)

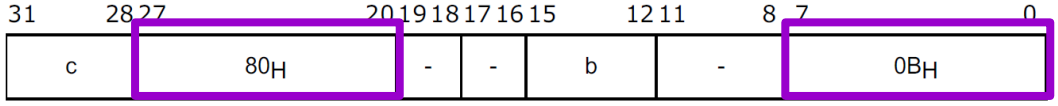
MOV D[c], D[b] (RR)



D[c] = D[b];

MOVE[c], D[b] (RR)

MOV E[c], D[b] (RR)



E[c] = sign_ext(D[b]);

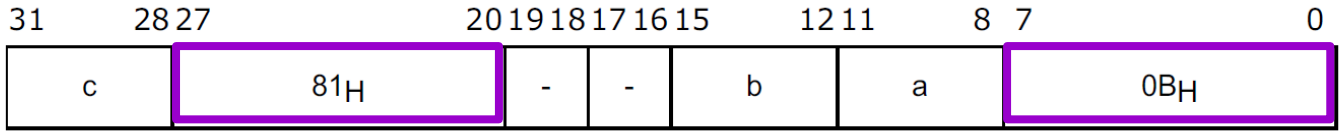
(2) 典型指令集

Integer Arithmetic（整数算术指令）

Move

MOVE[c], D[a], D[b] (RR)

MOV E[c], D[a], D[b] (RR)



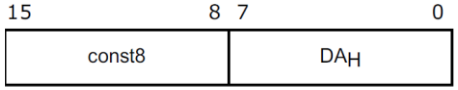
E[c] = {D[a], D[b]};

(2) 典型指令集

Integer Arithmetic（整数算术指令）

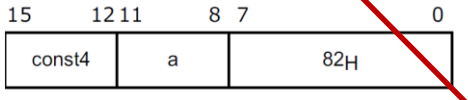
Move

MOVD[15], const8 (SC) **MOV D[15], const8 (SC)**



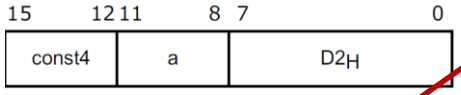
D[15] = zero_ext(const8);

MOVD[a], const4 (SRC)



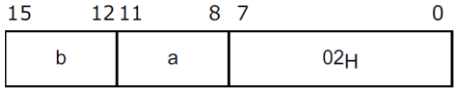
D[a] = sign_ext(const4);

MOVE[a], const4 (SRC)



E[a] = sign_ext(const4);

MOVD[a], D[b] (SRR)



D[a] = D[b];

Table 1-5 16-bit Opcode Formats

	15-14	13-12	11-10	09-08	07-06	05-04	03-02	01-00
SB	disp8				op1			
SBC	const4		disp4		op1			
SBR	s2		disp4		op1			
SBRN	n		disp4		op1			
SC	const8				op1			
SLR	s2		d		op1			
SLRO	off4		d		op1			
SR	op2		s1/d		op1			
SRC	const4		s1/d		op1			
SRO	s2		off4		op1			
SRR	s2		s1/d		op1			
SRRS	s2		s1/d		n	op1		
SSR	s2		s1		op1			
SSRO	off4		s1		op1			

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Move

```
mov    d3, d1
mov    d3, #-30000
mov    e0, d5; e0 = sign_ext(d5)
mov    e0, d6, d3; e0 = d6_d3 or d1=d6 d0=d3
mov    e0, 1234H; e0 = 0000000000001234H
```

#代表 立即数immediate value

; 代表 注释

```
mov    d1, d2
mov    d1, #6
mov    d15, #126
mov    e0, #EH; e0 = FFFFFFFFFFFFFFFEH
```

根据语法规定，此处数据类型为带符号4位整数的补码

带符号数以补码的形式保存，EH代表补码1110，真值为-2，化为64位的补码，为FFFFFFFFFFFFFFFEH

指令代码及含义

汇编语言、机器操作码和C语言RTL格式三种表达方式

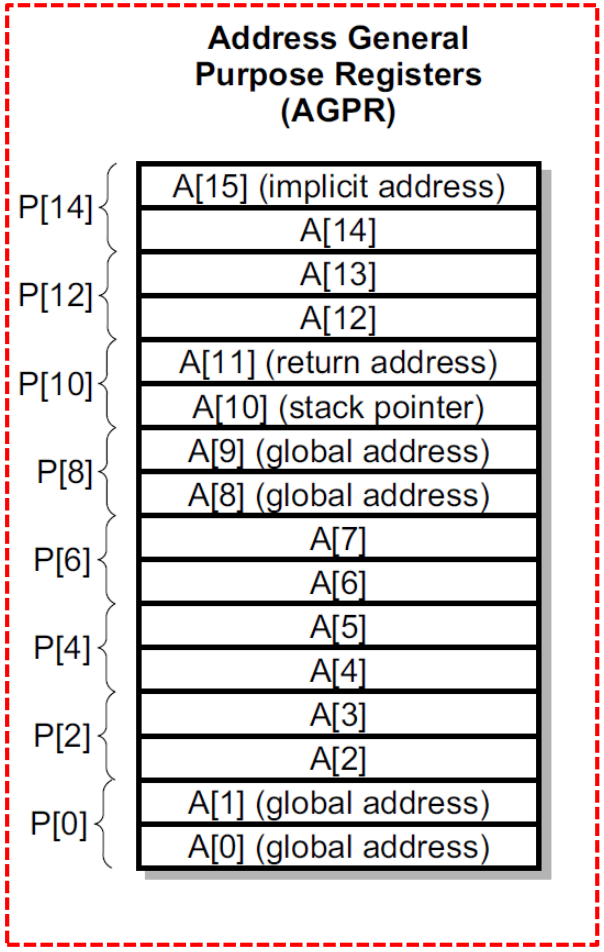
实际代码编写，比如寄存器A[10]应该写成a10

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Move

MOV.A
Move Value to Address Register



(2) 典型指令集

Integer Arithmetic（整数算术指令） | **Move**

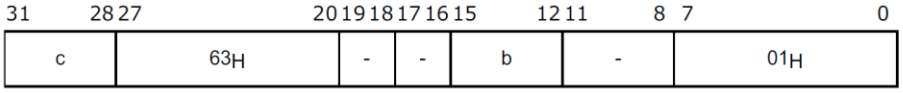
MOV.A
Move Value to Address Register

Description

Move the contents of data register D[b] to address register A[c].

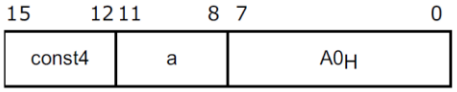
Move the contents of either data register D[b] (format SRR) or const4 (format SRC) to address register A[a]. The value const4 is zero-extended before it is moved.

MOV.AA[c], D[b] (RR)



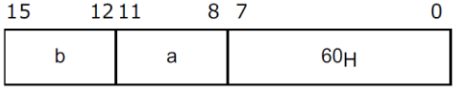
A[c] = D[b];

MOV.AA[a], const4 (SRC)



A[a] = zero_ext(const4);

MOV.AA[a], D[b] (SRR)



A[a] = D[b];

Examples

```
mov.a    a3, d1

mov.a    a4, d2
mov.a    a4, 7
```

(2) 典型指令集

Integer Arithmetic（整数算术指令） | **Move**

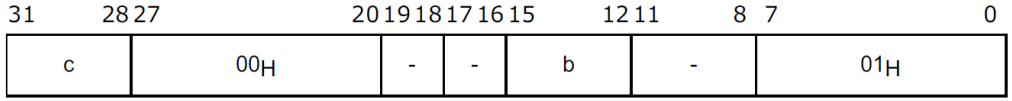
MOV.AA
Move Address from Address Register

Description

Move the contents of address register A[b] to address register A[c].

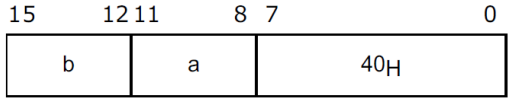
Move the contents of address register A[b] to address register A[a].

MOV.AAA[c], A[b] (RR)



A[c] = A[b];

MOV.AAA[a], A[b] (SRR)



A[a] = A[b];

Examples

```
mov.aa    a3, a4

mov.aa    a4, a2
```

(2) 典型指令集

Integer Arithmetic（整数算术指令） | **Move**

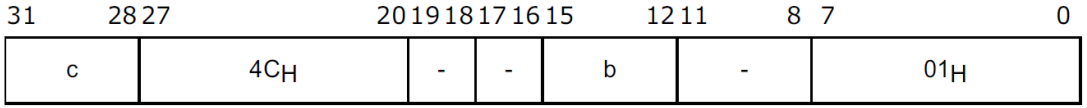
MOV.D
Move Address to Data Register

Description

Move the contents of address register A[b] to data register D[c].

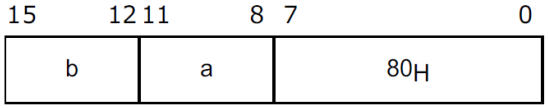
Move the contents of address register A[b] to data register D[a].

MOV.DD[c], A[b] (RR)



D[c] = A[b];

MOV.DD[a], A[b] (SRR)



D[a] = A[b];

Examples

```
mov.d    d3, a4

mov.d    d1, a2
```

（2）典型指令集

Integer Arithmetic（整数算术指令）

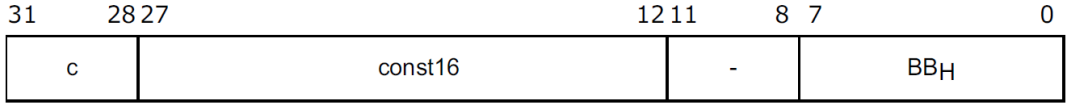
Move

MOV.U
Move Unsigned

Description

Move the zero-extended value const16 to data register D[c].

MOV.UD[c], const16 (RLC)



D[c] = zero_ext(const16);

Examples

```
mov.u    d3, #526
```

(2) 典型指令集

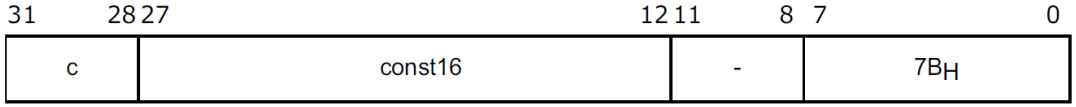
Integer Arithmetic（整数算术指令） | **Move**

MOVH
Move High

Description

Move the value const16 to the most-significant half-word of data register D[c] and set the least-significant 16-bits to zero.

MOVHD[c], const16 (RLC)



D[c] = {const16, 16'h0000};

Examples

```
movh    d3, #526
```

（2）典型指令集

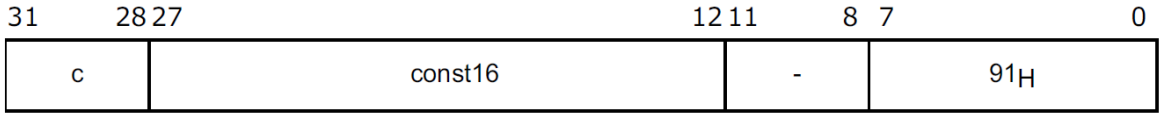
Integer Arithmetic（整数算术指令） | **Move**

MOVH.A
Move High to Address

Description

Move the value const16 to the most-significant half-word of address register A[c] and set the least-significant 16-bits to zero.

MOVH.AA[c], const16 (RLC)



A[c] = {const16, 16'h0000};

Examples

```
movh.a    a3, #526
```

（2）典型指令集

Integer Arithmetic（整数算术指令）

Addition and Subtraction

The addition instructions have three versions:

- ADD (No saturation)
- ADDS (Signed saturation)
- ADDS.U (Unsigned saturation)

ADD

Add

Description

Add the contents of data register D[a] to the contents of either data register D[b] (instruction format RR) or const9 (instruction format RC) and put the result in data register D[c]. The operands are treated as 32-bit integers, and the const9 value is sign-extended before the addition is performed.

Add the contents of either data register D[a] or D[15] to the contents of data register D[b] or const4, and put the result in either data register D[a] or D[15]. The operands are treated as 32-bit signed integers, and the const4 value is sign-extended before the addition is performed.

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Addition and Subtraction

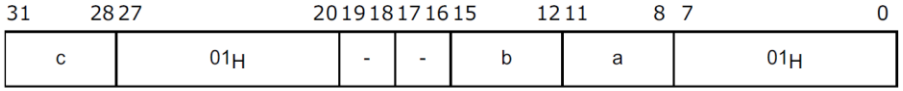
ADD.A
Add Address

Description

Add the contents of address register A[a] to the contents of address register A[b] and put the result in address register A[c].

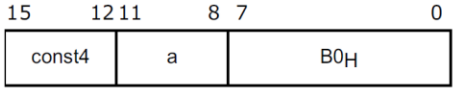
Add the contents of address register A[a] to the contents of either address register A[b] or const4 and put the result in address register A[a].

ADD.AA[c], A[a], A[b] (RR)



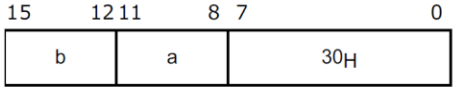
A[c] = A[a] + A[b];

ADD.AA[a], const4 (SRC)



A[a] = A[a] + sign_ext(const4);

ADD.AA[a], A[b] (SRR)



A[a] = A[a] + A[b];

Examples

```
add.a    a3, a4, a2

add.a    a1, a2
add.a    a3, 6
```

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Addition and Subtraction

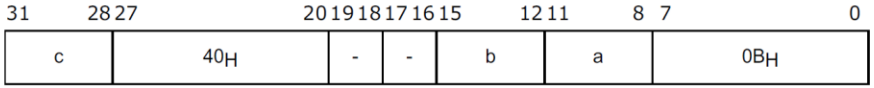
ADD.B
Add Packed Byte

ADD.H
Add Packed Half-word

Description

Add the contents of each byte (ADD.B) or half-word (ADD.H) of D[a] and D[b] and put the result in each corresponding byte or half-word of D[c]. The overflow condition is calculated for each byte or half-word of the packed quantity.

ADD.BD[c], D[a], D[b] (RR)



result_byte3 = D[a][31:24] + D[b][31:24];

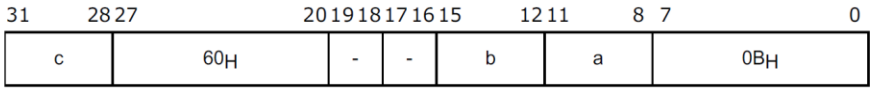
result_byte2 = D[a][23:16] + D[b][23:16];

result_byte1 = D[a][15:8] + D[b][15:8];

result_byte0 = D[a][7:0] + D[b][7:0];

D[c] = {result_byte3[7:0], result_byte2[7:0], result_byte1[7:0], result_byte0[7:0]};

ADD.HD[c], D[a], D[b] (RR)



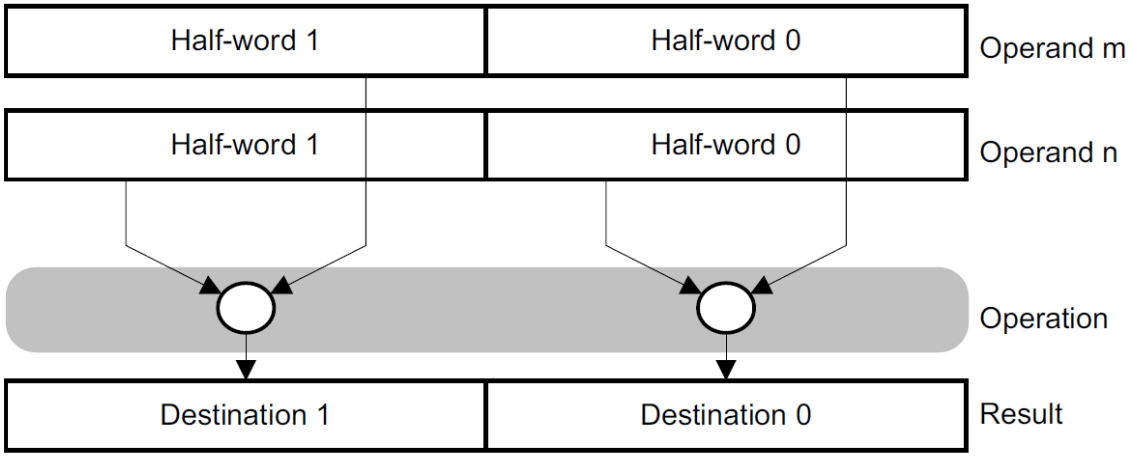
result_halfword1 = D[a][31:16] + D[b][31:16];

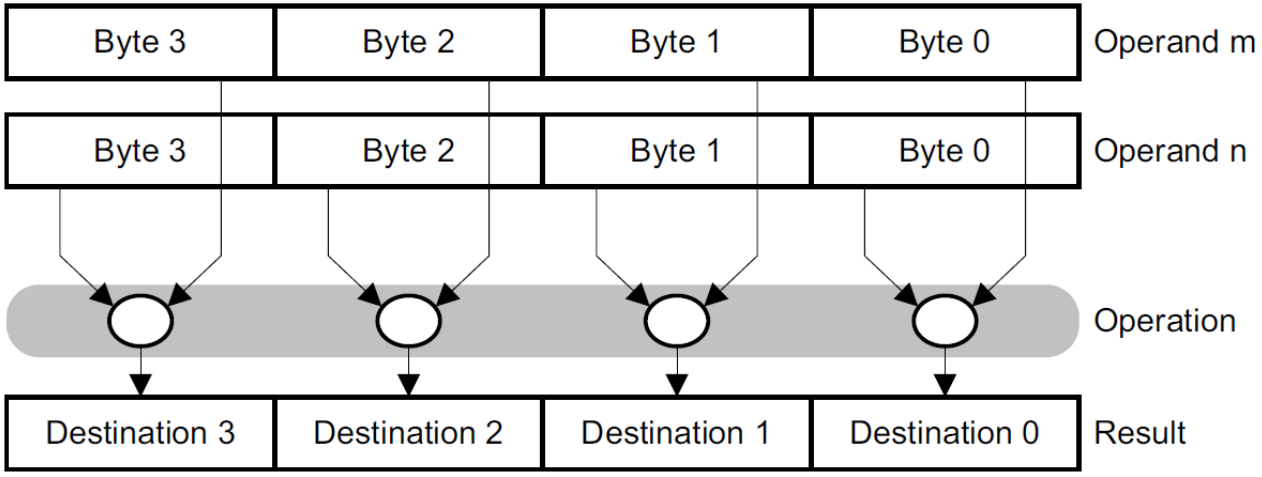
result_halfword0 = D[a][15:0] + D[b][15:0];

D[c] = {result_halfword1[15:0], result_halfword0[15:0]};

（2）典型指令集

Packed Arithmetic（包装算术指令）





(2) 典型指令集

Integer Arithmetic（整数算术指令）

Addition and Subtraction

ADDC
Add with Carry

ADDI
Add Immediate

ADDIH
Add Immediate High

ADDIH.A
Add Immediate High to Address

ADDS
Add Signed with Saturation

ADDS.H
Add Signed Packed Half-word with Saturation

ADDS.HU
Add Unsigned Packed Half-word with Saturation

ADDS.U
Add Unsigned with Saturation

ADDX
Add Extended

carry bit: 进位、移位

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Addition and Subtraction

SUB
Subtract

SUB.A
Subtract Address

SUB.B
Subtract Packed Byte

SUB.H
Subtract Packed Half-word

SUBC
Subtract With Carry

SUBS
Subtract Signed with Saturation

SUBS.U
Subtract Unsigned with Saturation

SUBS.H
Subtract Packed Half-word with Saturation

SUBS.HU
Subtract Packed Half-word Unsigned with Saturation

SUBX
Subtract Extended

All add instructions except those with constants, have similar corresponding subtract instructions. Because the immediate of ADDI is sign-extended, it may be used for both addition and subtraction

ADDI
Add Immediate

ADDIH
Add Immediate High

ADDIH.A
Add Immediate High to Address

(2) 典型指令集

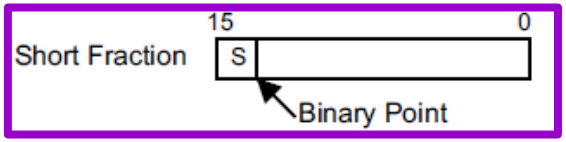
Integer Arithmetic（整数算术指令）

Multiply and Multiply-Add

Table 1-4 Data Type Modifiers

Data Type Modifier	Name	Description	Example
D	Data	32-bit data	MOV.D
D	Double-word	64-bit data/address	LD.D
W	Word	32-bit (word) data	EQ.W
A	Address	32-bit address	ADD.A
Q	Q Format	16-bit signed fraction (Q format)	MADD.Q
H	Half-word	16-bit data or two packed half-words	ADD.H
B	Byte	8-bit data or four packed bytes	ADD.B
T	Bit	1-bit data	AND.T
U	Unsigned	Unsigned data type	ADDS.U

Note: Q format can be used as signed half-word multipliers.



2.4 DSP Arithmetic

DSP arithmetic instructions operate on 16-bit signed fractional data in the 1.15 format (also known as Q15), and 32-bit signed fractional data in 1.31 format (Q31).

Data values in this format have a single, high-order sign bit, with a value of 0 or -1, followed by an implied binary point and fraction. Their values are in the range [-1, 1).

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Multiply and Multiply-Add

MUL
Multiply

MULS
Multiply, Saturated

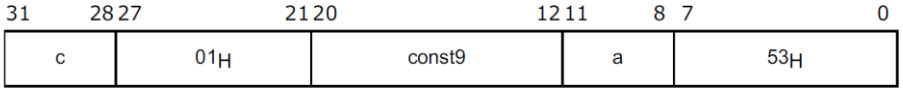
Description

Multiply two signed 32-bit integers and put the product into a 32-bit or 64-bit register. The value const9 is sign-extended before the multiplication is performed. The MULS result is saturated on overflow.

Multiply D[a] by D[b] (two signed 32-bit integers) and put the product into D[a].

MULD[c], D[a], const9 (RC)

(32 * K9) --> 32 signed

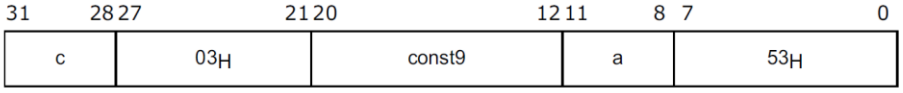


result = D[a] * sign_ext(const9);

D[c] = result[31:0];

MULE[c], D[a], const9 (RC)

(32 * K9) --> 64 signed



result = D[a] * sign_ext(const9);

E[c] = result[63:0];

Examples

```
mul    d3, d1, d2
mul    d2, d4, #21H
mul    e2, d5, d1
muls    d2, d0, d0
```

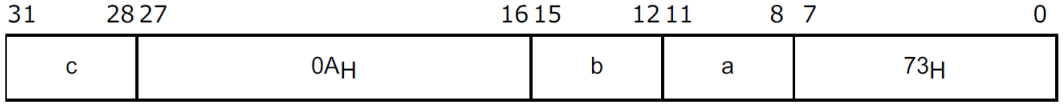
(2) 典型指令集

Integer Arithmetic（整数算术指令）

Multiply and Multiply-Add

MULD[c], D[a], D[b] (RR2)

(32 * 32) --> 32 signed

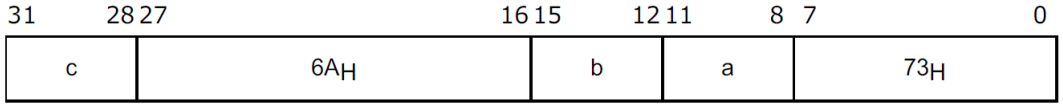


result = D[a] * D[b];

D[c] = result[31:0];

MULE[c], D[a], D[b] (RR2)

(32 * 32) --> 64 signed

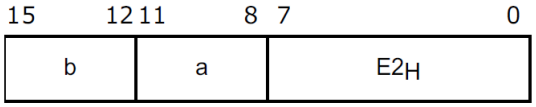


result = D[a] * D[b];

E[c] = result[63:0];

MULD[a], D[b] (SRR)

(32 * 32) --> 32 signed



result = D[a] * D[b];
D[a] = result[31:0];

(2) 典型指令集

Integer Arithmetic（整数算术指令）

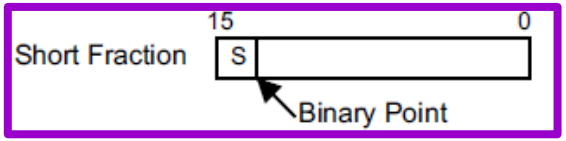
Multiply and Multiply-Add

MUL.Q
Multiply Q Format

Description

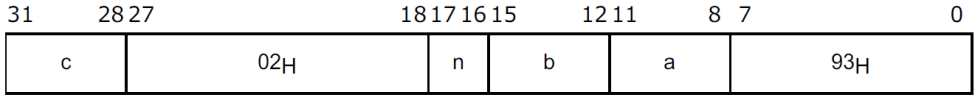
Multiply two signed 16-bit or 32-bit values and put the product (left justified if n == 1) into a 32-bit or 64-bit register. There are two cases of 16*16 operations, four cases of 16*32 operations and two cases of 32*32 operations.

Note that n should only take the values 0 or 1, any other value returns an undefined result. If (n == 1) then 8000_H * 8000_H = $7FFFFFFF_H$ (for signed 16-bit * 16-bit multiplications only).



MUL.QD[c], D[a], D[b], n (RR1)

(32 * 32)Up --> 32

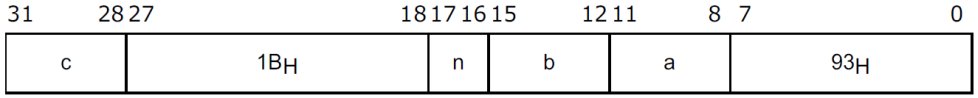


result = ((D[a] * D[b]) << n) >> 32;

D[c] = result[31:0]; // Fraction

MUL.QE[c], D[a], D[b], n (RR1)

(32 * 32) --> 64



result = (D[a] * D[b]) << n;

E[c] = result[63:0]; // Multi-precision fraction

Examples

```
mul.q   d3, d1U, d2U, #1
mul.q   d3, d1L, d2L, #1
mul.q   d3, d1, d2U, #0
mul.q   d3, d1, d2L, #1
mul.q   d2, d1, d2, #1
mul.q   e2, d1, d0U, #1
mul.q   e2, d1, d0L, #1
mul.q   e2, d1, d7, #1
```

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Multiply and Multiply-Add

MADD
Multiply-Add

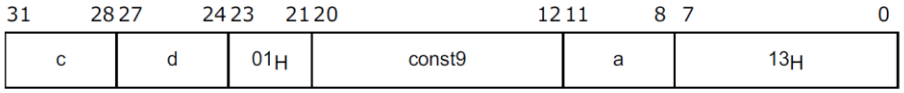
MADDS
Multiply-Add, Saturated

Description

Multiply two signed 32-bit integers, add the product to a signed 32-bit or 64-bit integer and put the result into a 32-bit or 64-bit register. The value const9 is sign-extended before the multiplication is performed. The MADDS result is saturated on overflow.

MADDD[c], D[d], D[a], const9 (RCR)

32 + (32 * K9)--> 32 signed

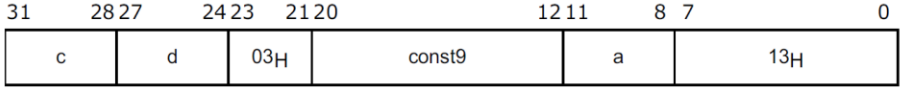


result = D[d] + (D[a] * sign_ext(const9));

D[c] = result[31:0];

MADDE[c], E[d], D[a], const9 (RCR)

64 + (32 * K9)--> 64 signed



result = E[d] + (D[a] * sign_ext(const9));

E[c] = result[63:0];

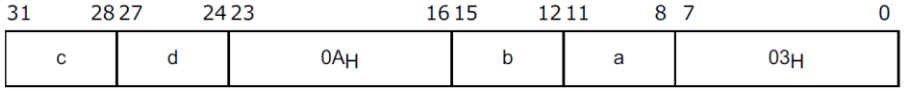
(2) 典型指令集

Integer Arithmetic（整数算术指令）

Multiply and Multiply-Add

MADDD[c], D[d], D[a], D[b] (RRR2)

32 + (32 * 32)--> 32 signed

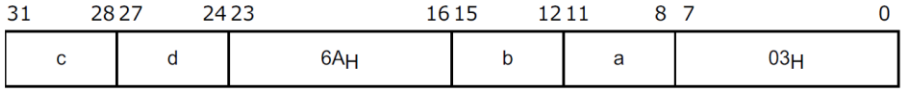


result = D[d] + (D[a] * D[b]);

D[c] = result[31:0];

MADDE[c], E[d], D[a], D[b] (RRR2)

64 + (32 * 32)--> 64 signed

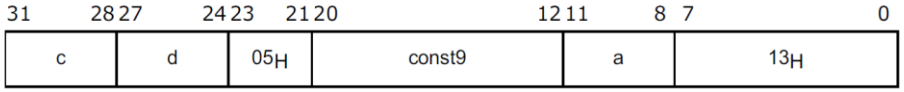


result = E[d] + (D[a] * D[b]);

E[c] = result[63:0];

MADDSD[c], D[d], D[a], const9 (RCR)

32 + (32 * K9)--> 32 signed saturated

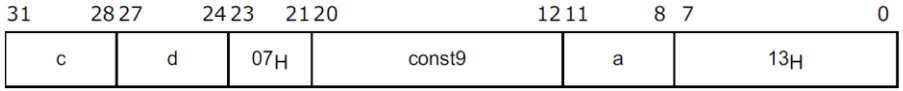


result = D[d] + (D[a] * sign_ext(const9));

D[c] = ssov(result, 32);

MADDSE[c], E[d], D[a], const9 (RCR)

64 + (32 * K9)--> 64 signed saturated

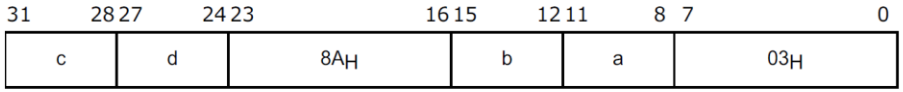


result = E[d] + (D[a] * sign_ext(const9));

E[c]= ssov(result, 64);

MADDSD[c], D[d], D[a], D[b] (RRR2)

32 + (32 * 32)--> 32 signed saturated

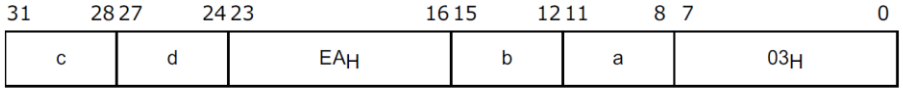


result = D[d] + (D[a] * D[b]);

D[c] = ssov(result, 32);

MADDSE[c], E[d], D[a], D[b] (RRR2)

64 + (32 * 32)--> 64 signed saturated



result = E[d] + (D[a] * D[b]);

E[c] = ssov(result, 64);

（2）典型指令集

Integer Arithmetic（整数算术指令）

Multiply and Multiply-Add

```
madd    d0, d1, d2, d3
madd    d0, d1, d2, #7
madd    e0, e2, d6, d11
madd    e0, e0, d3, #80
madds   d5, d1, d2, d2
madds   d11, d1, d2, #7
madds   e0, e2, d6, d11
madds   e8, e10, d3, #80
```

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Multiply and Multiply-Add

- **MADD** - Multiply-Add
- **MADDS** - Multiply-Add, Saturated
- **MADD.H** - Packed Multiply-Add Q Format
- **MADDS.H** - Packed Multiply-Add Q Format, Saturated
- **MADD.Q** - Multiply-Add Q Format
- **MADDS.Q** - Multiply-Add Q Format, Saturated
- **MADD.U** - Multiply-Add Unsigned
- **MADDS.U** - Multiply-Add Unsigned, Saturated
- **MADDM.H** - Packed Multiply-Add Q Format Multi-precision
- **MADDMS.H** - Packed Multiply-Add Q Format Multi-precision, Saturated
- **MADDR.H** - Packed Multiply-Add Q Format with Rounding
- **MADDRS.H** - Packed Multiply-Add Q Format with Rounding, Saturated
- **MADDR.Q** - Multiply-Add Q Format with Rounding
- **MADDRS.Q** - Multiply-Add Q Format with Rounding, Saturated
- **MADDSU.H** - Packed Multiply-Add/Subtract Q Format
- **MADDSUS.H** - Packed Multiply-Add/Subtract Q Format Saturated
- **MADDSUM.H** - Packed Multiply-Add/Subtract Q Format Multi-precision
- **MADDSUMS.H** - Packed Multiply-Add/Subtract Q Format Multi-precision Saturated
- **MADDSUR.H** - Packed Multiply-Add/Subtract Q Format with Rounding
- **MADDSURS.H** - Packed Multiply-Add/Subtract Q Format with Rounding Saturated

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Multiply and Multiply-Add

- **MSUB** - Multiply-Subtract
- **MSUBS** - Multiply-Subtract, Saturated
- **MSUB.H** - Packed Multiply-Subtract Q Format
- **MSUBS.H** - Packed Multiply-Subtract Q Format, Saturated
- **MSUB.Q** - Multiply-Subtract Q Format
- **MSUBS.Q** - Multiply-Subtract Q Format, Saturated
- **MSUB.U** - Multiply-Subtract Unsigned
- **MSUBS.U** - Multiply-Subtract Unsigned, Saturated
- **MSUBAD.H** - Packed Multiply-Subtract/Add Q Format
- **MSUBADS.H** - Packed Multiply-Subtract/Add Q Format, Saturated
- **MSUBADM.H** - Packed Multiply-Subtract/Add Q Format-Multi-precision
- **MSUBADMS.H** - Packed Multiply-Subtract/Add Q Format-Multi-precision, Saturated
- **MSUBADR.H** - Packed Multiply-Subtract/Add Q Format with Rounding
- **MSUBADRS.H** - Packed Multiply-Subtract/Add Q Format with Rounding, Saturated
- **MSUBM.H** - Packed Multiply-Subtract Q Format-Multi-precision
- **MSUBMS.H** - Packed Multiply-Subtract Q Format-Multi-precision, Saturated
- **MSUBR.H** - Packed Multiply-Subtract Q Format with Rounding
- **MSUBRS.H** - Packed Multiply-Subtract Q Format with Rounding, Saturated
- **MSUBR.Q** - Multiply-Subtract Q Format with Rounding
- **MSUBRS.Q** - Multiply-Subtract Q Format with Rounding, Saturated

（2）典型指令集

Integer Arithmetic（整数算术指令）

Multiply and Multiply-Add

- **MUL** - Multiply
- **MULS** - Multiply, Saturated
- **MUL.H** - Packed Multiply Q Format
- **MUL.Q** - Multiply Q Format
- **MUL.U** - Multiply Unsigned
- **MULS.U** - Multiply Unsigned, Saturated
- **MULM.H** - Packed Multiply Q Format-Multi-precision
- **MULMS.H** - MULMS.H (DEPRECATED)
- **MULR.H** - Packed Multiply Q Format with Rounding
- **MULR.Q** - Multiply Q Format with Rounding

DSP Arithmetic（数字信号处理指令）

（2）典型指令集

Integer Arithmetic（整数算术指令） | **Division**

DIV
Divide

DIV.U
Divide Unsigned

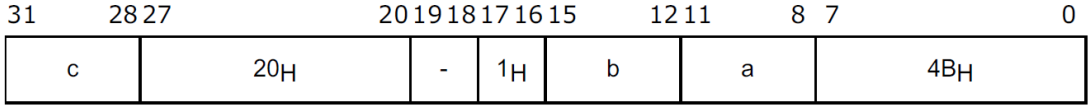
Description

Divide the contents of register D[a] by the contents of register D[b]. Put the resulting quotient in E[c][31:0] and the remainder in E[c][63:32].

The operands and results are treated as signed 32-bit integers for the DIV instruction and as unsigned 32-bit integers for the DIV.U instruction.

Overflow occurs if the divisor (D[b]) is zero. For DIV, Overflow also occurs if the dividend (D[a]) is the maximum negative number and the divisor is minus 1.

DIVE[c], D[a], D[b] (RR)



（2）典型指令集

Integer Arithmetic（整数算术指令）

Division

- **DVADJ** - Divide-Adjust
- **DIV** - Divide
- **DIV.U** - Divide Unsigned
- **DVINIT** - Divide-Initialization Word
- **DVINIT.U** - Divide-Initialization Word Unsigned
- **DVINIT.B** - Divide-Initialization Byte
- **DVINIT.BU** - Divide-Initialization Byte Unsigned
- **DVINIT.H** - Divide-Initialization Half-word
- **DVINIT.HU** - Divide-Initialization Half-word Unsigned
- **DVSTEP** - Divide-Step
- **DVSTEP.U** - Divide-Step Unsigned

DSP Arithmetic（数字信号处理指令）

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Absolute Value, Absolute Difference	ABS, ABSDIF
Min, Max, Saturate	MAX, MIN, SAT
Conditional Arithmetic Instructions	CADD, CADDN, CSUB, CSUBN, SEL, SELN
Logical	AND, OR, NAND, NOR, XOR, XNOR, ANDN, ORN
Count Leading Zeros, Ones and Signs	CLZ, CLO, CLS
Shift	SH, SHA, SHAS
Bit-Field Extract and Insert	EXTR, EXTR.U, DEXTR, INSERT

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Absolute Value, Absolute Difference

ABS　ABSDIF

- **ABS** - Absolute Value
- **ABS.B** - Absolute Value Packed Byte
- **ABS.H** - Absolute Value Packed Half-word
- **ABSDIF** - Absolute Value of Difference
- **ABSDIF.B** - Absolute Value of Difference Packed Byte
- **ABSDIF.H** - Absolute Value of Difference Packed Half-word
- **ABSDIFS** - Absolute Value of Difference with Saturation
- **ABSDIFS.H** - Absolute Value of Difference Packed Half-word with Saturation
- **ABSS** - Absolute Value with Saturation
- **ABSS.H** - Absolute Value Packed Half-word with Saturation

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Min, Max, Saturate

MAX **MIN** **SAT**

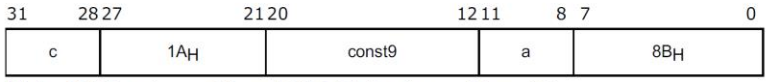
MAX
Maximum Value

MAX.U
Maximum Value Unsigned

Description

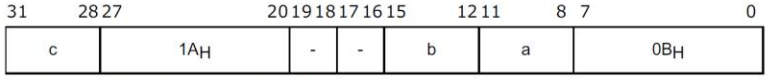
If the contents of data register D[a] are greater than the contents of either data register D[b] (instruction format RR) or const9 (instruction format RC), then put the contents of D[a] in data register D[c]; otherwise put the contents of either D[b] (format RR) or const9 (format RC) in D[c]. The operands are treated as either signed (MAX) or unsigned (MAX.U) 32-bit integers.

MAXD[c], D[a], const9 (RC)



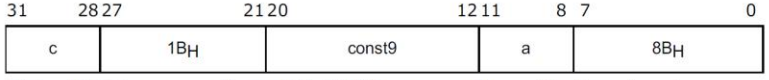
D[c] = (D[a] > sign_ext(const9)) ? D[a] : sign_ext(const9);

MAXD[c], D[a], D[b] (RR)



D[c] = (D[a] > D[b]) ? D[a] : D[b];

MAX.UD[c], D[a], const9 (RC)



D[c] = (D[a] > zero_ext(const9)) ? D[a] : zero_ext(const9); // unsigned

Examples

```
max   d3, d1, d2
max   d3, d1, #126
```

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Min, Max, Saturate

MAX **MIN** **SAT**

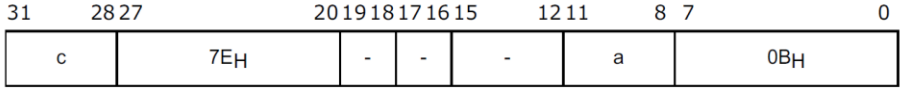
SAT.H
Saturate Half-word

Description

If the signed 32-bit value in D[a] is less than -32,768, then store the value -32,768 in D[c]. If D[a] is greater than 32,767, then store the value 32,767 in D[c]. Otherwise copy D[a] to D[c].

If the signed 32-bit value in D[a] is less than -32,768, then store the value -32,768 in D[a]. If D[a] is greater than 32,767, then store the value 32,767 in D[a]. Otherwise leave the contents of D[a] unchanged.

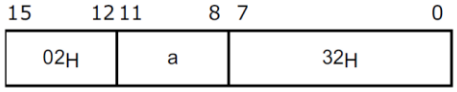
SAT.HD[c], D[a] (RR)



sat_neg = (D[a] < -8000$_H$) ? -8000$_H$: D[a];

D[c] = (sat_neg > 7FFF$_H$) ? 7FFF$_H$: sat_neg;

SAT.HD[a] (SR)



sat_neg = (D[a] < -8000$_H$) ? -8000$_H$: D[a];
D[a] = (sat_neg > 7FFF$_H$) ? 7FFF$_H$: sat_neg;

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Min, Max, Saturate

MAX **MIN** **SAT**

- **MAX** - Maximum Value
- **MAX.U** - Maximum Value Unsigned
- **MAX.B** - Maximum Value Packed Byte
- **MAX.BU** - Maximum Value Packed Byte Unsigned
- **MAX.H** - Maximum Value Packed Half-word
- **MAX.HU** - Maximum Value Packed Half-word Unsigned
- **MIN** - Minimum Value
- **MIN.U** - Minimum Value Unsigned
- **MIN.B** - Minimum Value Packed Byte
- **MIN.BU** - Minimum Value Packed Byte Unsigned
- **MIN.H** - Minimum Value Packed Half-word
- **MIN.HU** - Minimum Value Packed Half-word Unsigned
- **SAT.B** - Saturate Byte
- **SAT.BU** - Saturate Byte Unsigned
- **SAT.H** - Saturate Half-word
- **SAT.HU** - Saturate Half-word Unsigned

（2）典型指令集

Integer Arithmetic（整数算术指令）

Min, Max, Saturate

MAX | MIN | SAT

IXMAX
Find Maximum Index

IXMAX.U
Find Maximum Index (unsigned)

Description

Enables a search of maximum value and its related index in a vector of 16-bit signed (IXMAX) or unsigned (IXMAX.U) values.

The IXMAX and IXMAX.U instructions are not available in the TriCore 1.2 Architecture.

For all operations:

- E[d][15:0] Working index.
- E[d][31:16] Current index of maximum.
- E[d][47:32] Current value of maximum.
- E[d][63:48] 00_H.
- D[b][15:0] First compare value.
- D[b][31:16] Second compare value.
- E[c][15:0] Update working index.
- E[c][31:16] Update index of maximum.
- E[c][47:32] Update value of maximum.
- E[c][63:48] 00_H.

```
ixmax      e2, e8, d6
ixmax.u    e2, e0, d4
```

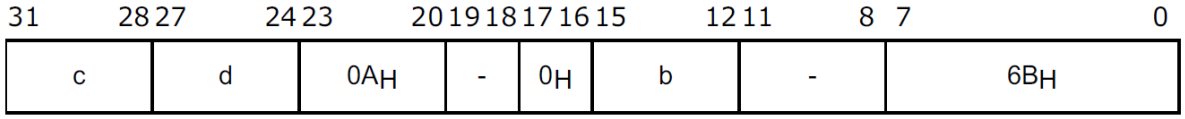
（2）典型指令集

Integer Arithmetic（整数算术指令）

Min, Max, Saturate

MAX **MIN** **SAT**

IXMAXE[c], E[d], D[b] (RRR)



E[c][15:0] = E[d][15:0] + 2;

E[c][63:48] = 00_H;

if (D[b][15:0] >= D[b][31:16]) AND (D[b][15:0] > E[d][47:32]) then {

E[c][47:32] = D[b][15:0];

E[c][31:16] = E[d][15:0];

} else if (D[b][31:16] > D[b][15:0]) AND (D[b][31:16] > E[d][47:32]) then {

E[c][47:32] = D[b][31:16];

E[c][31:16] = E[d][15:0]+1;

} else {

E[c][47:32] = E[d][47:32];

E[c][31:16] = E[d][31:16];

}

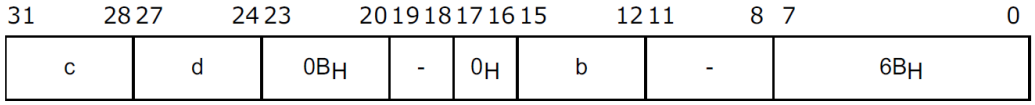
(2) 典型指令集

Integer Arithmetic（整数算术指令）

Min, Max, Saturate

MAX MIN SAT

IXMAX.UE[c], E[d], D[b] (RRR)



For IXMAX.U, the comparison is on unsigned numbers.

E[c][15:0] = E[d][15:0] + 2;

E[c][63:48] = 00_H;

if (D[b][15:0] >= D[b][31:16]) AND (D[b][15:0] > E[d][47:32]) then {

E[c][47:32] = D[b][15:0];

E[c][31:16] = E[d][15:0];

} else if (D[b][31:16] > D[b][15:0]) AND (D[b][31:16] > E[d][47:32]) then {

E[c][47:32] = D[b][31:16];

E[c][31:16] = E[d][15:0]+1;

} else {

E[c][47:32] = E[d][47:32];

E[c][31:16] = E[d][31:16];

}

For all index additions, on overflow: wrapping, no trap.

If the 1st compare value and 2nd compare value and current maximum value are the same, the priority select is: current maximum is the highest priority then 1st compare value and 2nd compare value is in the lowest priority.

（2）典型指令集

Integer Arithmetic（整数算术指令）

Conditional Arithmetic Instructions

CADD | CADDN | CSUB | CSUBN

SEL | SELN

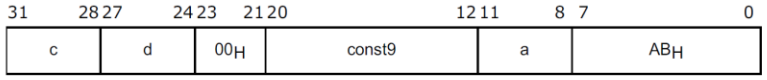
CADD
Conditional Add

Description

If the contents of data register D[d] are non-zero, then add the contents of data register D[a] and the contents of either register D[b] (instruction format RRR) or const9 (instruction format RCR) and put the result in data register D[c]; otherwise put contents of D[a] in D[c]. The const9 value is sign-extended.

If the contents of data register D[15] are non-zero, then add contents of data register D[a] and the contents of const4 and put the result in data register D[a]; otherwise the contents of D[a] is unchanged. The const4 value is sign-extended.

CADDD[c], D[d], D[a], const9 (RCR)

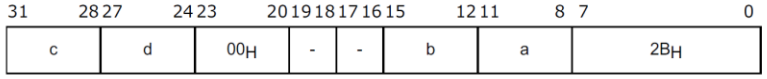


condition = D[d] != 0;

result = ((condition) ? D[a] + sign_ext(const9) : D[a]);

D[c] = result[31:0];

CADDD[c], D[d], D[a], D[b] (RRR)



condition = (D[d] != 0);

result = ((condition) ? D[a] + D[b] : D[a]);

D[c] = result[31:0];

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Conditional Arithmetic Instructions

CADD | CADDN | CSUB | CSUBN

SEL | SELN

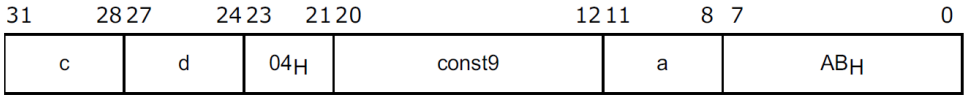
SEL
Select

Description

If the contents of data register D[d] are non-zero, copy the contents of data register D[a] to data register D[c]; otherwise copy the contents of either D[b] (instruction format RRR) or const9 (instruction format RCR), to D[c].

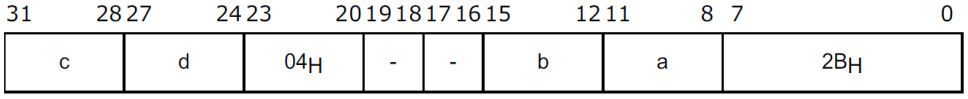
The value const9 (instruction format RCR) is sign-extended.

SELD[c], D[d], D[a], const9 (RCR)



D[c] = ((D[d] != 0) ? D[a] : sign_ext(const9));

SELD[c], D[d], D[a], D[b] (RRR)



D[c] = ((D[d] != 0) ? D[a] : D[b]);

（2）典型指令集

Integer Arithmetic（整数算术指令）

Logical | AND | OR | NAND | NOR | XOR | XNOR | ANDN | ORN

AND

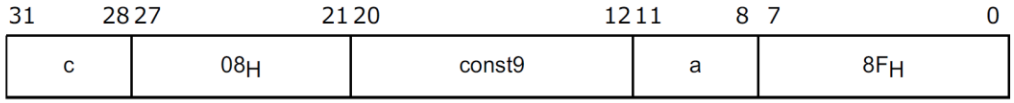
Bitwise AND

Description

Compute the bitwise AND of the contents of data register D[a] and the contents of either data register D[b] (instruction format RR) or const9 (instruction format RC) and put the result in data register D[c]. The const9 value is zero-extended.

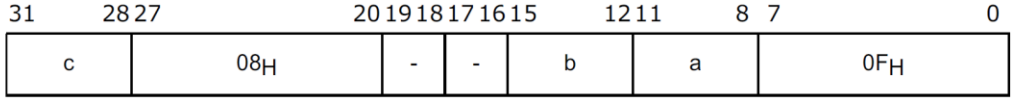
Compute the bitwise AND of the contents of either data register D[a] (instruction format SRR) or D[15] (instruction format SC) and the contents of either data register D[b] (format SRR) or const8 (format SC), and put the result in data register D[a] (format SRR) or D[15] (format SC). The const8 value is zero-extended.

ANDD[c], D[a], const9 (RC)



D[c] = D[a] & zero_ext(const9);

ANDD[c], D[a], D[b] (RR)



D[c] = D[a] & D[b];

C语言AND：逻辑运算
C语言&：位运算
汇编语言AND：是位运算还是逻辑运算，看情况

Integer Arithmetic（整数算术指令）

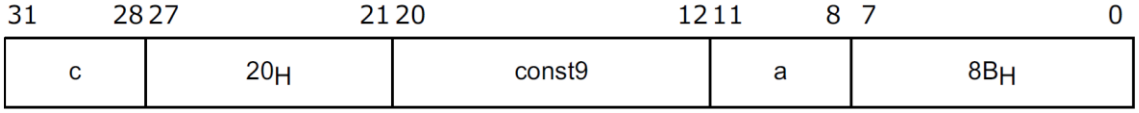
Logical | AND | OR | NAND | NOR | XOR | XNOR | ANDN | ORN

AND.EQ
Equal Accumulating

Description

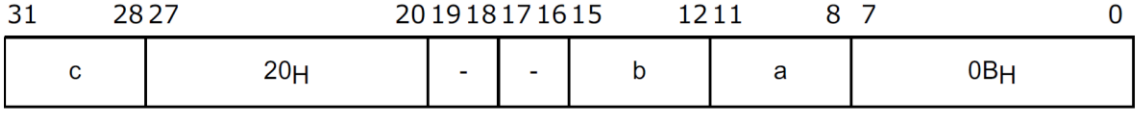
Compute the logical AND of D[c][0] and the boolean result of the equality comparison operation on the contents of data register D[a] and either data register D[b] (instruction format RR) or const9 (instruction RC). Put the result in D[c][0]. All other bits in D[c] are unchanged. The const9 value is sign-extended.

AND.EQD[c], D[a], const9 (RC)



D[c] = {D[c][31:1], D[c][0] AND (D[a] == sign_ext(const9))};

AND.EQD[c], D[a], D[b] (RR)



D[c] = {D[c][31:1], D[c][0] AND (D[a] == D[b])};

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Logical | AND | OR | NAND | NOR | XOR | XNOR | ANDN | ORN

- **AND** - Bitwise AND
- **AND.AND.T** - Accumulating Bit Logical AND-AND
- **AND.ANDN.T** - Accumulating Bit Logical AND-AND-Not
- **AND.NOR.T** - Accumulating Bit Logical AND-NOR
- **AND.OR.T** - Accumulating Bit Logical AND-OR
- **AND.EQ** - Equal Accumulating
- **AND.GE** - Greater Than or Equal Accumulating
- **AND.GE.U** - Greater Than or Equal Accumulating Unsigned
- **AND.LT** - Less Than Accumulating
- **AND.LT.U** - Less Than Accumulating Unsigned
- **AND.NE** - Not Equal Accumulating
- **AND.T** - Bit Logical AND
- **ANDN** - Bitwise AND-Not
- **ANDN.T** - Bit Logical AND-Not

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Logical | AND | OR | NAND | NOR | XOR | XNOR | ANDN | ORN

- **NAND** - Bitwise NAND
- **NAND.T** - Bit Logical NAND
- **NE** - Not Equal
- **NOR** - Bitwise NOR
- **NOR.T** - Bit Logical NOR
- **NOT (16-bit)** - Bitwise Complement NOT (16-bit)
- **OR** - Bitwise OR
- **OR.AND.T** - Accumulating Bit Logical OR-AND
- **OR.ANDN.T** - Accumulating Bit Logical OR-AND-Not
- **OR.NOR.T** - Accumulating Bit Logical OR-NOR
- **OR.OR.T** - Accumulating Bit Logical OR-OR
- **OR.EQ** - Equal Accumulating
- **OR.GE** - Greater Than or Equal Accumulating
- **OR.GE.U** - Greater Than or Equal Accumulating Unsigned
- **OR.LT** - Less Than Accumulating
- **OR.LT.U** - Less Than Accumulating Unsigned
- **OR.NE** - Not Equal Accumulating
- **OR.T** - Bit Logical OR
- **ORN** - Bitwise OR-Not
- **ORN.T** - Bit Logical OR-Not
- **XNOR** - Bitwise XNOR
- **XNOR.T** - Bit Logical XNOR
- **XOR** - Bitwise XOR
- **XOR.EQ** - Equal Accumulating
- **XOR.GE** - Greater Than or Equal Accumulating
- **XOR.GE.U** - Greater Than or Equal Accumulating Unsigned
- **XOR.LT** - Less Than Accumulating
- **XOR.LT.U** - Less Than Accumulating Unsigned
- **XOR.NE** - Not Equal Accumulating
- **XOR.T** - Bit Logical XOR

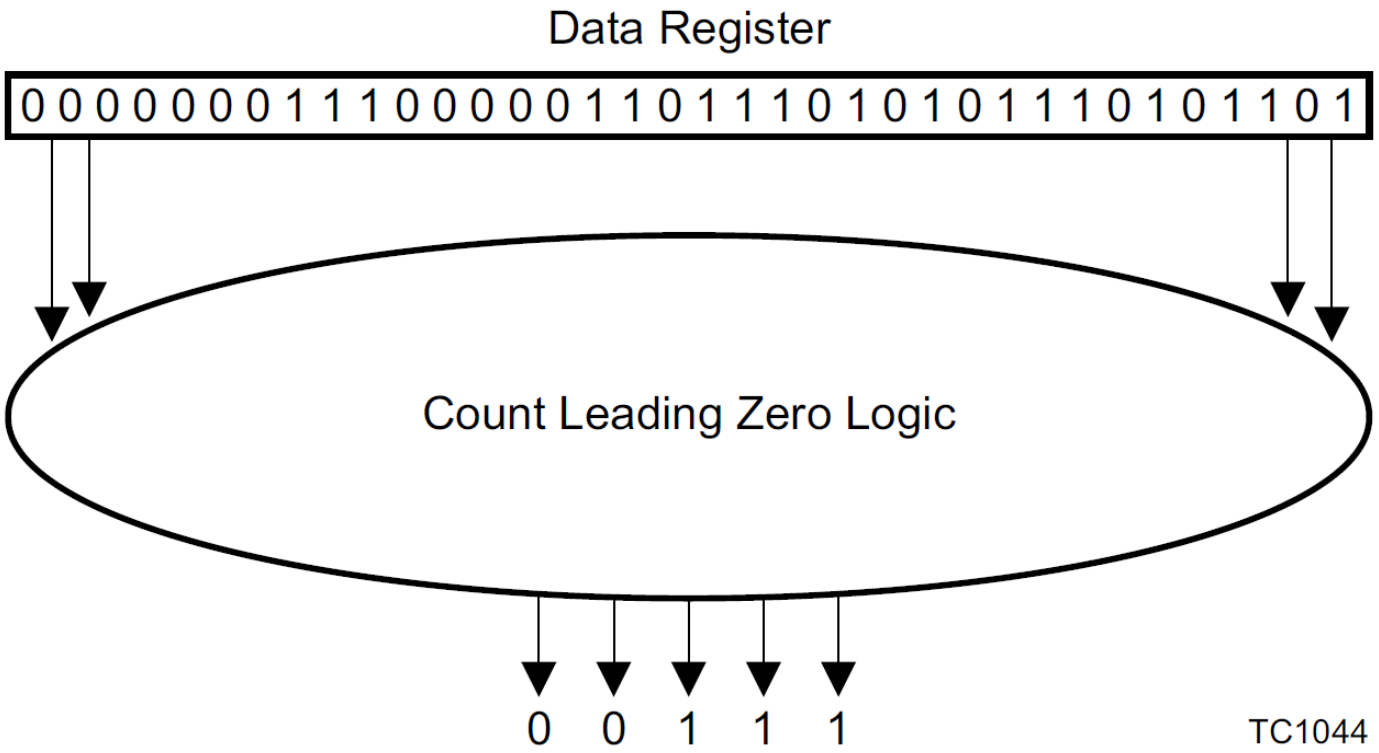
只有当运算符独立出现，或者操作数修饰符为T时，属于位运算bitwise

(2) 典型指令集

Integer Arithmetic（整数算术指令）

Count Leading Zeros, Ones and Signs

CLZ　CLO　CLS



(2) 典型指令集

DSP Arithmetic（数字信号处理指令）

MUL、MADD、MSUB、DIV系列指令

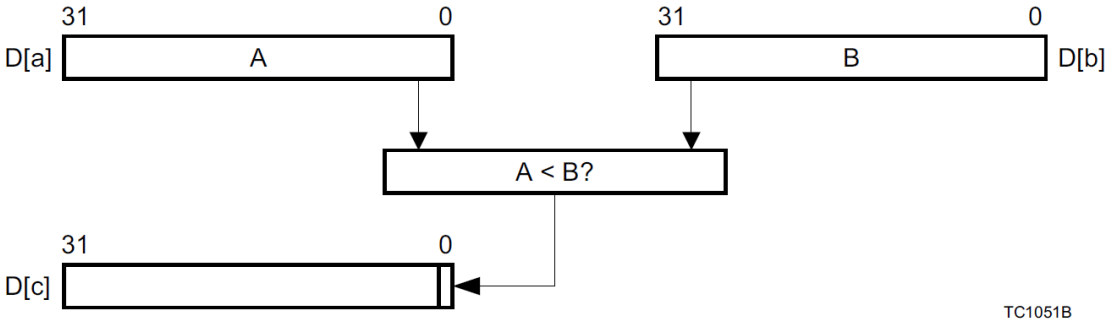
- **MADD** - Multiply-Add
- **MADDS** - Multiply-Add, Saturated
- **MADD.H** - Packed Multiply-Add Q Format
- **MADDS.H** - Packed Multiply-Add Q Format, Saturated
- **MADD.Q** - Multiply-Add Q Format
- **MADDS.Q** - Multiply-Add Q Format, Saturated
- **MADD.U** - Multiply-Add Unsigned
- **MADDS.U** - Multiply-Add Unsigned, Saturated
- **MADDM.H** - Packed Multiply-Add Q Format Multi-precision
- **MADDMS.H** - Packed Multiply-Add Q Format Multi-precision, Saturated
- **MADDR.H** - Packed Multiply-Add Q Format with Rounding
- **MADDRS.H** - Packed Multiply-Add Q Format with Rounding, Saturated
- **MADDR.Q** - Multiply-Add Q Format with Rounding
- **MADDRS.Q** - Multiply-Add Q Format with Rounding, Saturated
- **MADDSU.H** - Packed Multiply-Add/Subtract Q Format
- **MADDSUS.H** - Packed Multiply-Add/Subtract Q Format Saturated
- **MADDSUM.H** - Packed Multiply-Add/Subtract Q Format Multi-precision
- **MADDSUMS.H** - Packed Multiply-Add/Subtract Q Format Multi-precision Saturated
- **MADDSUR.H** - Packed Multiply-Add/Subtract Q Format with Rounding
- **MADDSURS.H** - Packed Multiply-Add/Subtract Q Format with Rounding Saturated

Compare Instructions（比较指令）

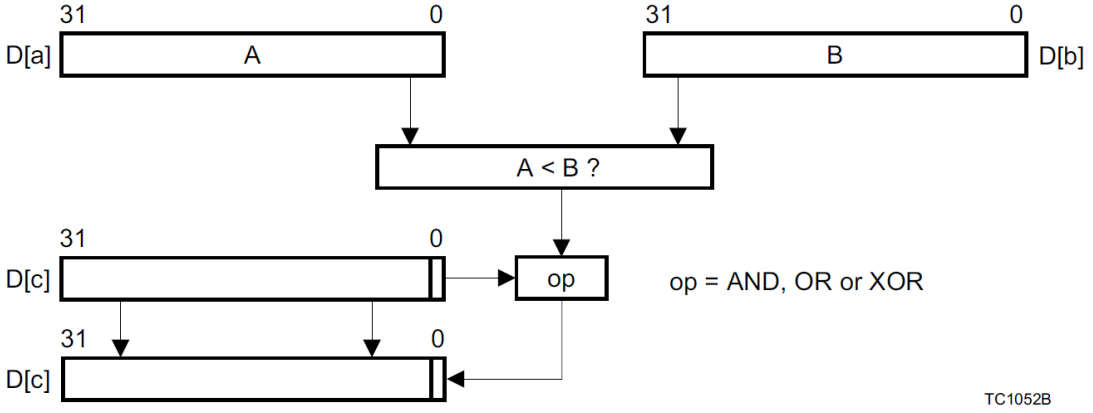
Simple Compare

The comparison instructions are:

- EQ (Equal)
- NE (Not Equal)
- LT (Less Than)
- GE (Greater than or Equal to)



Accumulating Compare

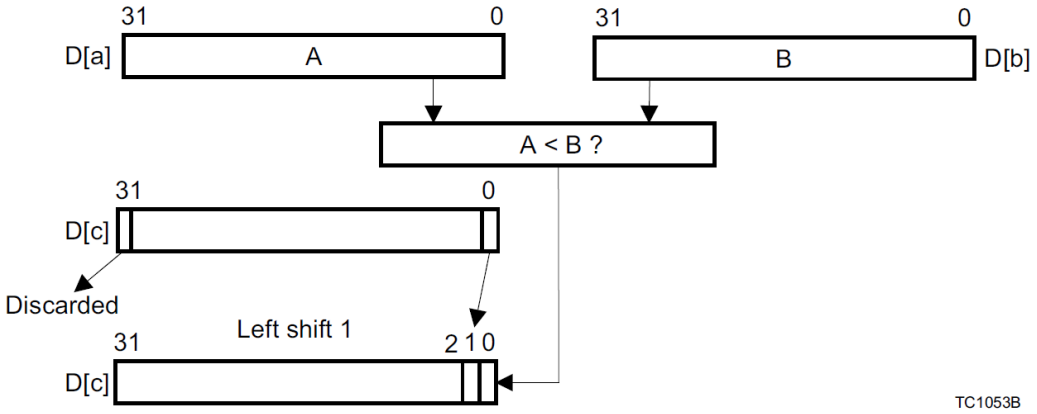


- **EQ** - Equal
- **EQ.B** - Equal Packed Byte
- **EQ.H** - Equal Packed Half-word
- **EQ.W** - Equal Packed Word
- **EQANY.B** - Equal Any Byte
- **EQANY.H** - Equal Any Half-word

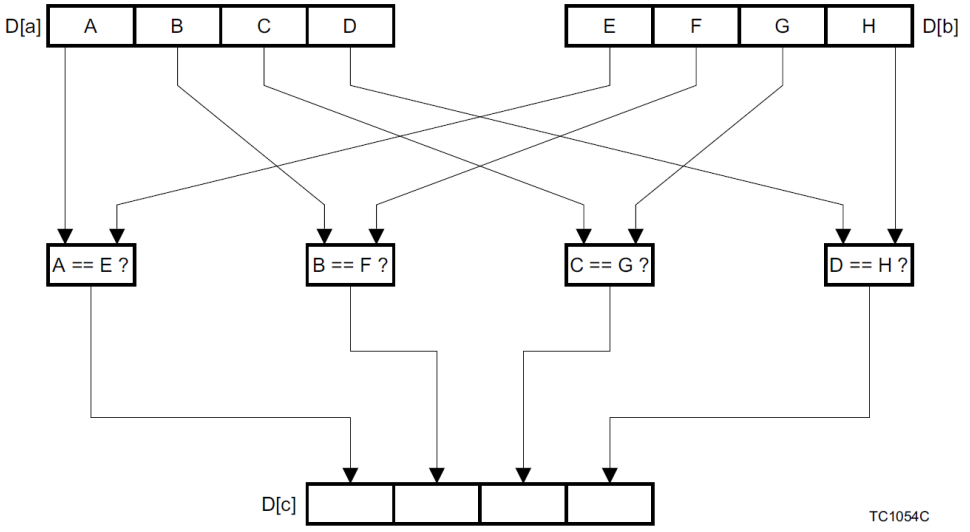
(2) 典型指令集

Compare Instructions（比较指令）

Compare with Shift



Packed Compare

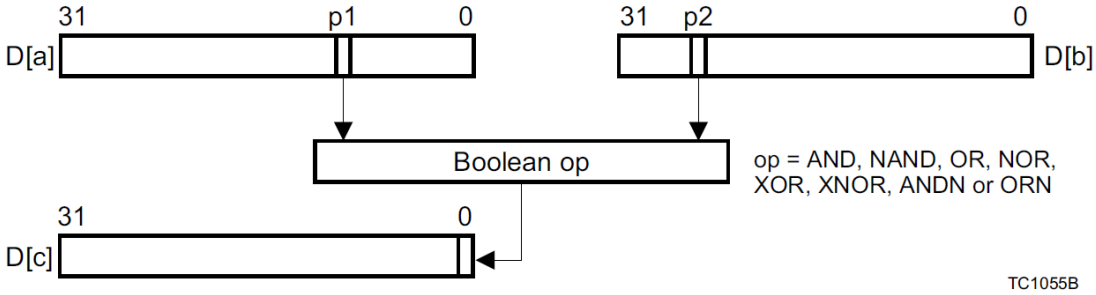


(2) 典型指令集

Bit Operations（位操作指令）

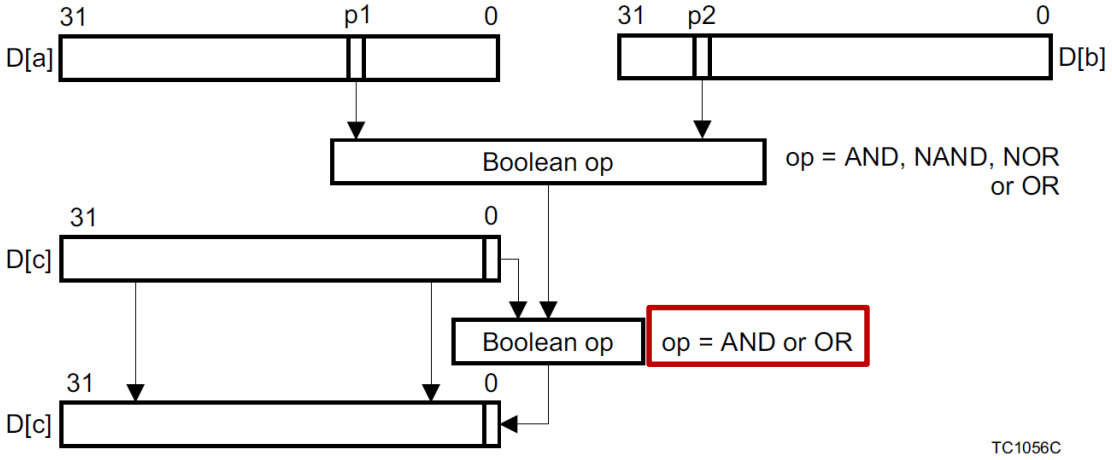
Simple Bit Operations

- AND
- NAND
- OR
- NOR
- XOR
- XNOR
- ANDN
- ORN



Accumulating Bit Operations

- AND.AND.T
- AND.ANDN.T
- AND.NOR.T
- AND.OR.T
- OR.AND.T
- OR.ANDN.T
- OR.NOR.T
- OR.OR.T

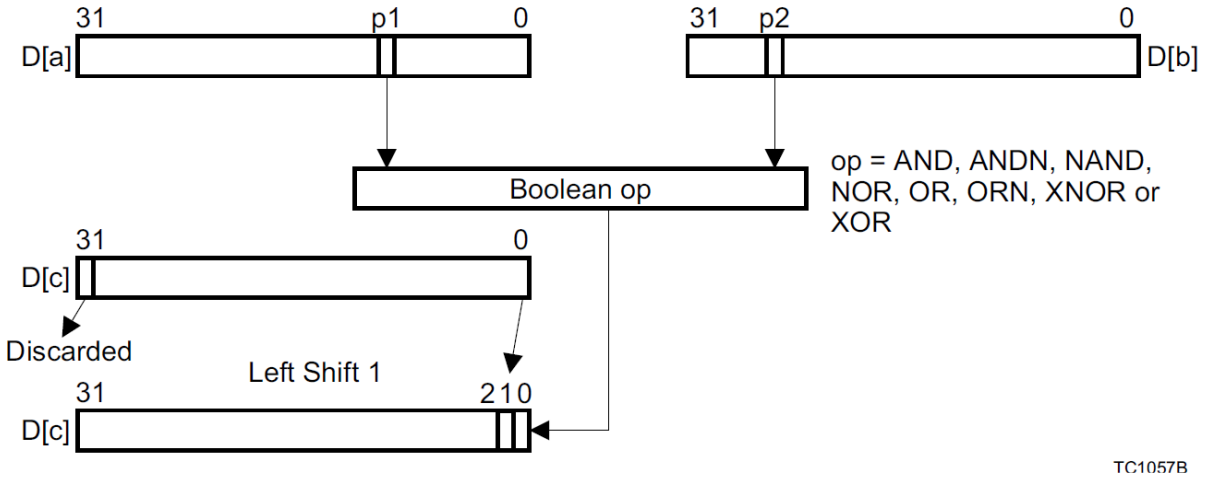


(2) 典型指令集

Bit Operations（位操作指令）

Shifting Bit Operations

SH.AND.T



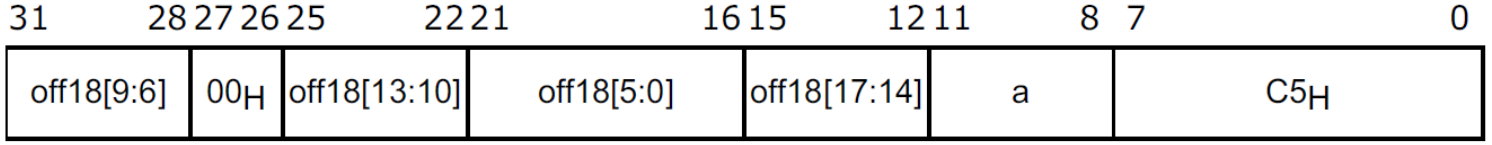
- **SH** - Shift
- **SH.EQ** - Shift Equal
- **SH.GE** - Shift Greater Than or Equal
- **SH.GE.U** - Shift Greater Than or Equal Unsigned
- **SH.H** - Shift Packed Half-words
- **SH.LT** - Shift Less Than
- **SH.LT.U** - Shift Less Than Unsigned
- **SH.NE** - Shift Not Equal
- **SH.AND.T** - Accumulating Shift-AND
- **SH.ANDN.T** - Accumulating Shift-AND-Not
- **SH.NAND.T** - Accumulating Shift-NAND
- **SH.NOR.T** - Accumulating Shift-NOR
- **SH.OR.T** - Accumulating Shift-OR
- **SH.ORN.T** - Accumulating Shift-OR-Not
- **SH.XNOR.T** - Accumulating Shift-XNOR
- **SH.XOR.T** - Accumulating Shift-XOR
- **SHA** - Arithmetic Shift
- **SHA.H** - Arithmetic Shift Packed Half-words
- **SHAS** - Arithmetic Shift with Saturation

（2）典型指令集

Address Arithmetic（地址运算指令）

Load Effective Address (LEA)

LEAA[a], off18 (ABS)(Absolute Addressing Mode)



EA = {off18[17:14], 14b'0, off18[13:0]};

A[a] = EA[31:0];

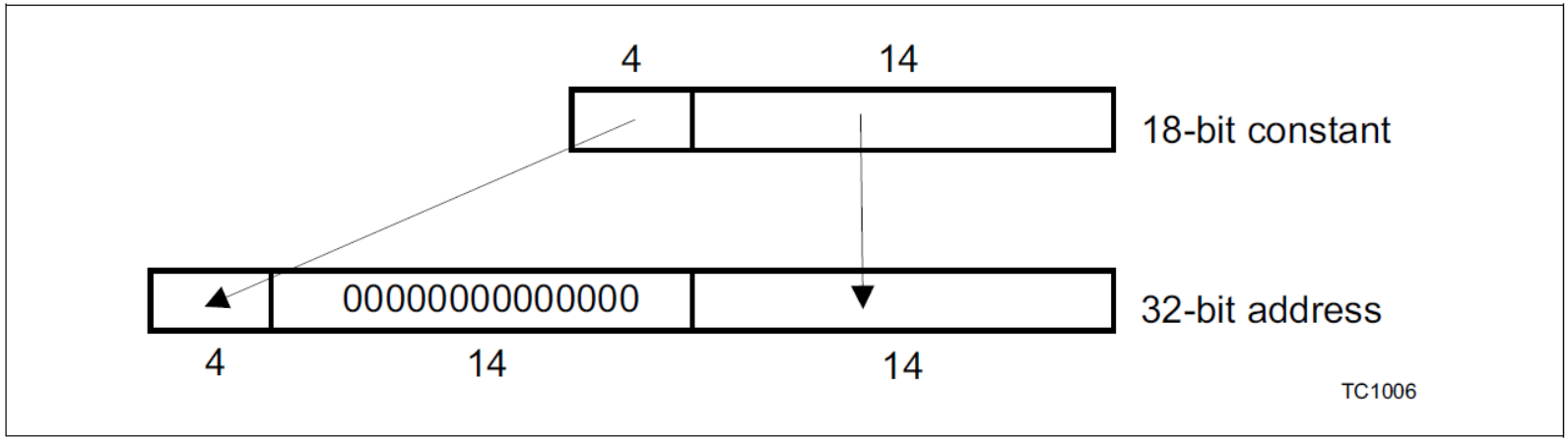


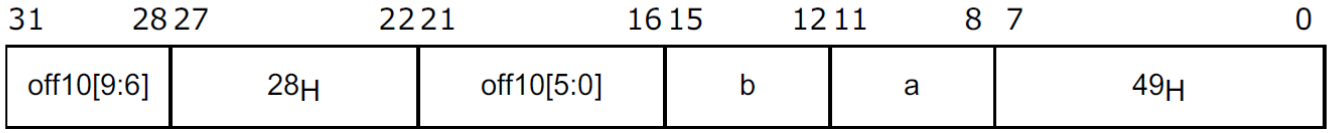
Figure 2-3 Translation of Absolute Address to Full Effective Address

（2）典型指令集

Address Arithmetic（地址运算指令）

Load Effective Address (LEA)

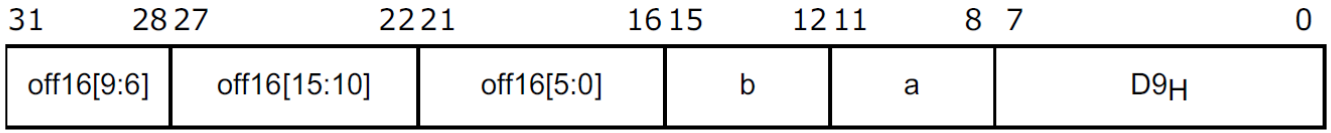
LEAA[a], A[b], off10 (BO)(Base + Short Offset Addressing Mode)



EA = A[b] + sign_ext(off10);

A[a] = EA[31:0];

LEAA[a], A[b], off16 (BOL)(Base + Long Offset Addressing Mode)



EA = A[b] + sign_ext(off16);

A[a] = EA[31:0];

（2）典型指令集

Address Comparison（地址比较指令）

Less Than (LT.A)

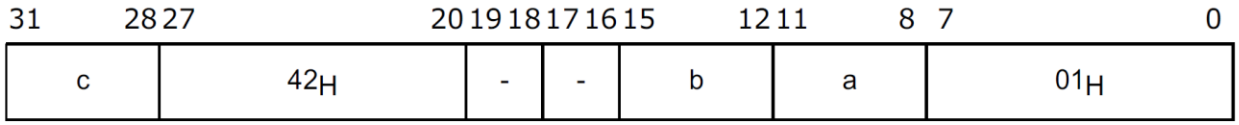
- **LT** - Less Than
- **LT.U** - Less Than Unsigned
- **LT.B** - Less Than Packed Byte
- **LT.BU** - Less Than Packed Byte Unsigned
- **LT.H** - Less Than Packed Half-word
- **LT.HU** - Less Than Packed Half-word Unsigned
- **LT.W** - Less Than Packed Word
- **LT.WU** - Less Than Packed Word Unsigned

LT.A
Less Than Address

Description

If the contents of address register A[a] are less than the contents of address register A[b], set the least-significant bit of D[c] to one and clear the remaining bits to zero; otherwise clear all bits in D[c]. The operands are treated as unsigned 32-bit integers.

LT.AD[c], A[a], A[b] (RR)



D[c] = (A[a] < A[b]); // unsigned

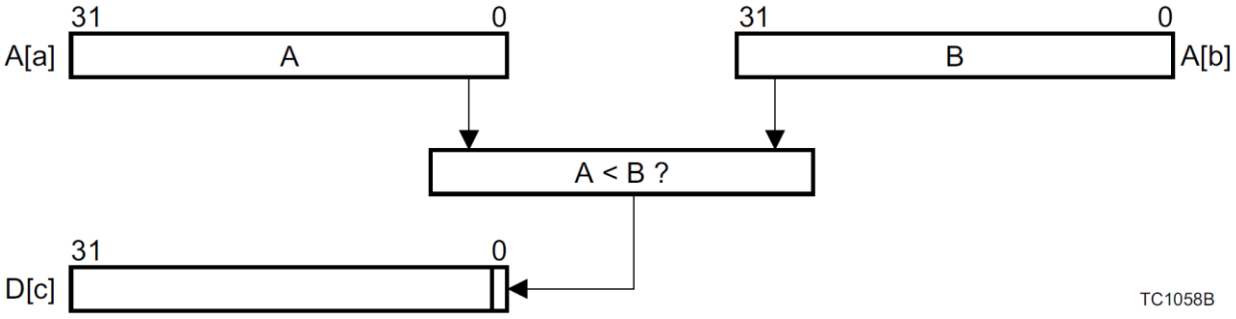
(2) 典型指令集

Address Comparison（地址比较指令）

Less Than (LT.A)　　**EQZ.A**　　**NEZ.A**

Table 2-2　Operation Equivalents

Implicit Comparison Operation		TriCore Equivalent Comparison Operation	
LE.A	D[c], A[a], A[b]	GE.A	D[c], A[b], A[a]
GT.A	D[c], A[a], A[b]	LT.A	D[c], A[b], A[a]



- **EQ.A** - Equal to Address
- **EQZ.A** - Equal Zero Address
- **NE.A** - Not Equal Address
- **NEZ.A** - Not Equal Zero Address

(2) 典型指令集

Branch Instructions（跳转指令）

- Jump instructions
- Jump and Link instructions
- Call and Return instructions

无条件跳转指令

J

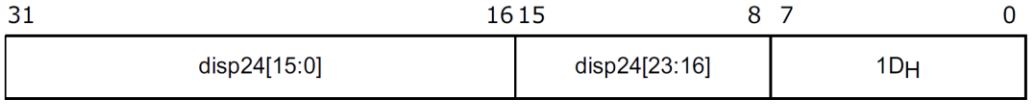
Jump Unconditional

Description

Add the value specified by disp24, sign-extended and multiplied by 2, to the contents of PC and jump to that address.

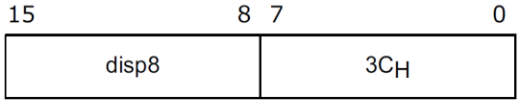
Add the value specified by disp8, sign-extended and multiplied by 2, to the contents of PC and jump to that address.

Jdisp24 (B)



PC = PC + sign_ext(disp24) * 2;

Jdisp8 (SB)



PC = PC + sign_ext(disp8) * 2;

（2）典型指令集

Branch Instructions（跳转指令）

- Jump instructions
- Jump and Link instructions
- Call and Return instructions

无条件跳转指令

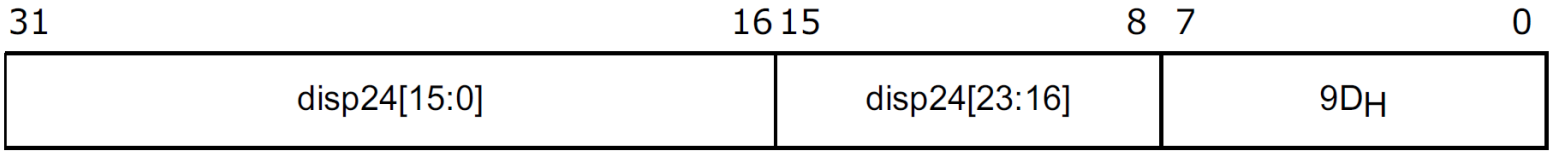
JA
Jump Unconditional Absolute

Description

Load the value specified by disp24 into PC and jump to that address.

The value disp24 is used to form the Effective Address (EA).

JAdisp24 (B)



PC = {disp24[23:20], 7'b0000000, disp24[19:0], 1'b0};

（2）典型指令集

Branch Instructions（跳转指令）

- Jump instructions
- Jump and Link instructions
- Call and Return instructions

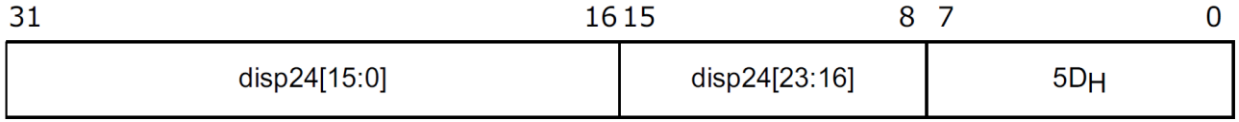
无条件跳转指令

JL
Jump and Link

Description

Store the address of the next instruction in A[11] (return address). Add the value specified by disp24, sign-extended and multiplied by 2, to the contents of PC and jump to that address.

JLdisp24 (B)



A[11] = PC + 4;

PC = PC + sign_ext(disp24) * 2;

（2）典型指令集

Branch Instructions（跳转指令）

- Jump instructions
- Jump and Link instructions
- Call and Return instructions

无条件跳转指令

CALL
Call

Description

Add the value specified by disp24, multiplied by two and sign-extended, to the address of the CALL instruction and jump to the resulting address. The target address range is ±16 MBytes relative to the current PC. In parallel with the jump, save the caller's Upper Context to an available Context Save Area (CSA). Set register A[11] (return address) to the address of the next instruction beyond the call.

Note: After CALL, upper context registers are undefined except for A[10] and A[11].

Note: When the PSW is saved, the CDE bit is forced to '1'.

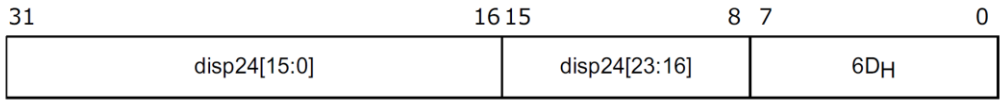
（2）典型指令集

Branch Instructions（跳转指令）

- Jump instructions
- Jump and Link instructions
- Call and Return instructions

无条件跳转指令

CALLdisp24 (B)




```
if (FCX == 0) trap(FCU);
if (PSW.CDE) then if (cdc_increment()) then trap(CDO);
PSW.CDE = 1;
ret_addr = PC + 4;
tmp_FCX = FCX;
EA = {FCX.FCXS, 6'b0, FCX.FCXO, 6'b0};
new_FCX = M(EA, word);
M(EA,16 * word) = {PCXI, PSW, A[10], A[11], D[8], D[9], D[10], D[11], A[12], A[13], A[14], A[15], D[12], D[13],
D[14], D[15]};
PCXI.PCPN = ICR.CCPN;
PCXI.PIE = ICR.IE;
PCXI.UL = 1;
PCXI[19:0] = FCX[19:0];
FCX[19:0] = new_FCX[19:0];
PC = PC + sign_ext(2 * disp24);
A[11] = ret_addr[31:0];
if (tmp_FCX == LCX) trap(FCD);
```

（2）典型指令集

Branch Instructions（跳转指令）

- Jump instructions
- Jump and Link instructions
- Call and Return instructions

无条件跳转指令

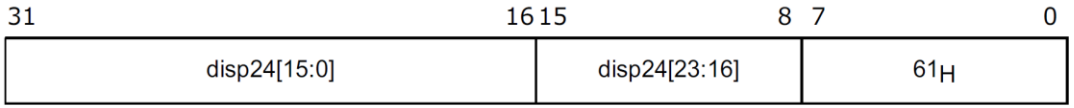
FCALL
Fast Call

Description

Add the value specified by disp24, multiplied by two and sign-extended to 32-bits, to the address of the FCALL instruction and jump to the resulting address. The target address range is ±16 MBytes relative to the current PC.

Store A[11] to the memory address specified by A[10] pre-decremented by 4. Store the address of the next instruction in A[11].

FCALLdisp24 (B)




```
ret_addr = PC + 4;
EA = A[10] - 4;
M(EA,word) = A[11];
PC = PC + sign_ext(2 * disp24);
A[11] = ret_addr[31:0];
A[10] = EA[31:0];
```

（2）典型指令集

Branch Instructions（跳转指令）

- Jump instructions
- Jump and Link instructions
- Call and Return instructions

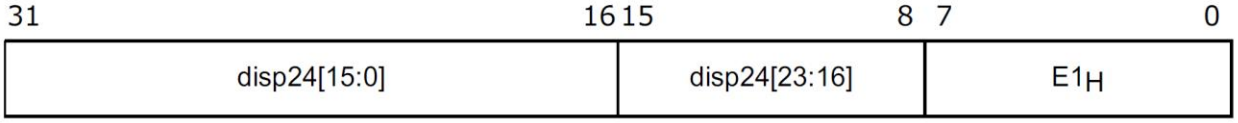
无条件跳转指令

FCALLA
Fast Call Absolute

Description

Jump to the address specified by disp24. Store A[11] to the memory address specified by A[10] pre-decremented by 4. Store the address of the next instruction in A[11].

FCALLAdisp24 (B)




```
ret_addr = PC + 4;
EA = A[10] - 4;
M(EA,word) = A[11];
PC = {disp24[23:20], 7'b0, disp24[19:0], 1'b0};
A[11] = ret_addr[31:0];
A[10] = EA[31:0]
```

(2) 典型指令集

Branch Instructions（跳转指令）

- Jump instructions
- Jump and Link instructions
- Call and Return instructions

无条件跳转指令

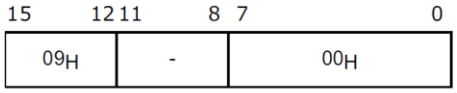
RET
Return from Call

Description

Return from a function that was invoked with a CALL instruction. The return address is in register A[11] (return address). The caller's upper context register values are restored as part of the return operation.

Return from a function that was invoked with a CALL instruction. The return address is in register A[11] (return address). The caller's upper context register values are restored as part of the return operation.

RET(SR)




```
if (PSW.CDE) then if (cdc_decrement()) then trap(CDU);
if (PCXI[19:0] == 0) then trap(CSU);
if (PCXI.UL == 0) then trap(CTYP);
PC = {A[11] [31:1], 1'b0};
EA = {PCXI.PCXS, 6'b0, PCXI.PCXO, 6'b0};
{new_PCXI, new_PSW, A[10], A[11], D[8], D[9], D[10], D[11], A[12], A[13], A[14], A[15], D[12], D[13], D[14],
D[15]} = M(EA, 16 * word);
M(EA, word) = FCX;
FCX[19:0] = PCXI[19:0];
PCXI = new_PCXI;
PSW = {new_PSW[31:26], PSW[25:24], new_PSW[23:0]};
```

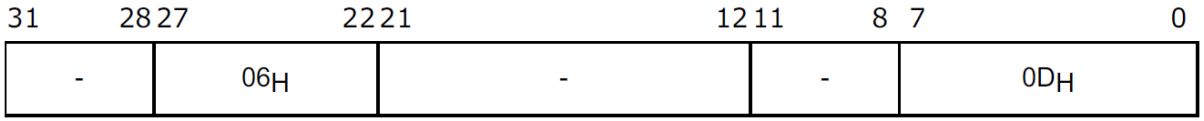
(2) 典型指令集

Branch Instructions（跳转指令）

- Jump instructions
- Jump and Link instructions
- Call and Return instructions

无条件跳转指令

RET(SYS)



if (PSW.CDE) then if (cdc_decrement()) then trap(CDU);

if (PCXI[19:0] == 0) then trap(CSU);

if (PCXI.UL == 0) then trap(CTYP);

PC = {A[11] [31:1], 1'b0};

EA = {PCXI.PCXS, 6'b0, PCXI.PCXO, 6'b0};

{new_PCXI, new_PSW, A[10], A[11], D[8], D[9], D[10], D[11], A[12], A[13], A[14], A[15], D[12], D[13], D[14], D[15]} = M(EA, 16 * word);

M(EA, word) = FCX;

FCX[19:0] = PCXI[19:0];

PCXI = new_PCXI;

PSW = {new_PSW[31:26], PSW[25:24], new_PSW[23:0]};

Branch Instructions（跳转指令）

无条件跳转指令

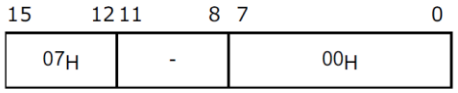
FRET
Return from Fast Call

Description

Return from a function that was invoked with an FCALL instruction. Jump to the address specified by A[11]. Load A[11] from the address specified by A[10] then increment A[10] by 4.

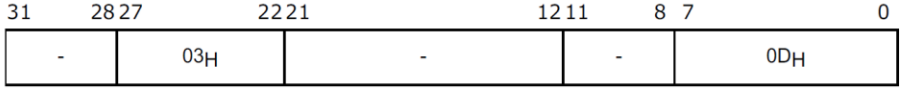
Return from a function that was invoked with an FCALL instruction. Jump to the address specified by A[11]. Load A[11] from the address specified by A[10] then increment A[10] by 4.

FRET (SR)




```
PC = {A[11] [31:1], 1'b0};
EA = A[10];
A[11] = M(EA, word);
A[10] = A[10] + 4;
```

FRET (SYS)




```
PC = {A[11] [31:1], 1'b0};
EA = A[10];
A[11] = M(EA, word);
A[10] = A[10] + 4;
```

（2）典型指令集

Branch Instructions（跳转指令）

Conditional Branch
条件跳转指令

- **JEQ** - Jump if Equal
- **JGE** - Jump if Greater Than or Equal
- **JGE.U** - Jump if Greater Than or Equal Unsigned
- **JGEZ (16-bit)** - Jump if Greater Than or Equal to Zero (16-bit)
- **JGTZ (16-bit)** - Jump if Greater Than Zero (16-bit)
- **JLEZ (16-bit)** - Jump if Less Than or Equal to Zero (16-bit)
- **JLT** - Jump if Less Than
- **JLT.U** - Jump if Less Than Unsigned
- **JLTZ (16-bit)** - Jump if Less Than Zero (16-bit)
- **JNE** - Jump if Not Equal
- **JNED** - Jump if Not Equal and Decrement
- **JNEI** - Jump if Not Equal and Increment
- **JNZ (16-bit)** - Jump if Not Equal to Zero (16-bit)
- **JNZ.T** - Jump if Not Equal to Zero Bit
- **JZ (16-bit)** - Jump if Zero (16-bit)
- **JZ.T** - Jump if Zero Bit

（2）典型指令集

Branch Instructions（跳转指令）

Loop Instructions循环指令

- **LOOP** - Loop
- **LOOPU** - Loop Unconditional

Load and Store Instructions（装载与存储指令）

2.10 Load and Store Instructions

The load (LD.x) and store (ST.x) instructions move data between registers and memory using seven addressing modes (Table 2-3).

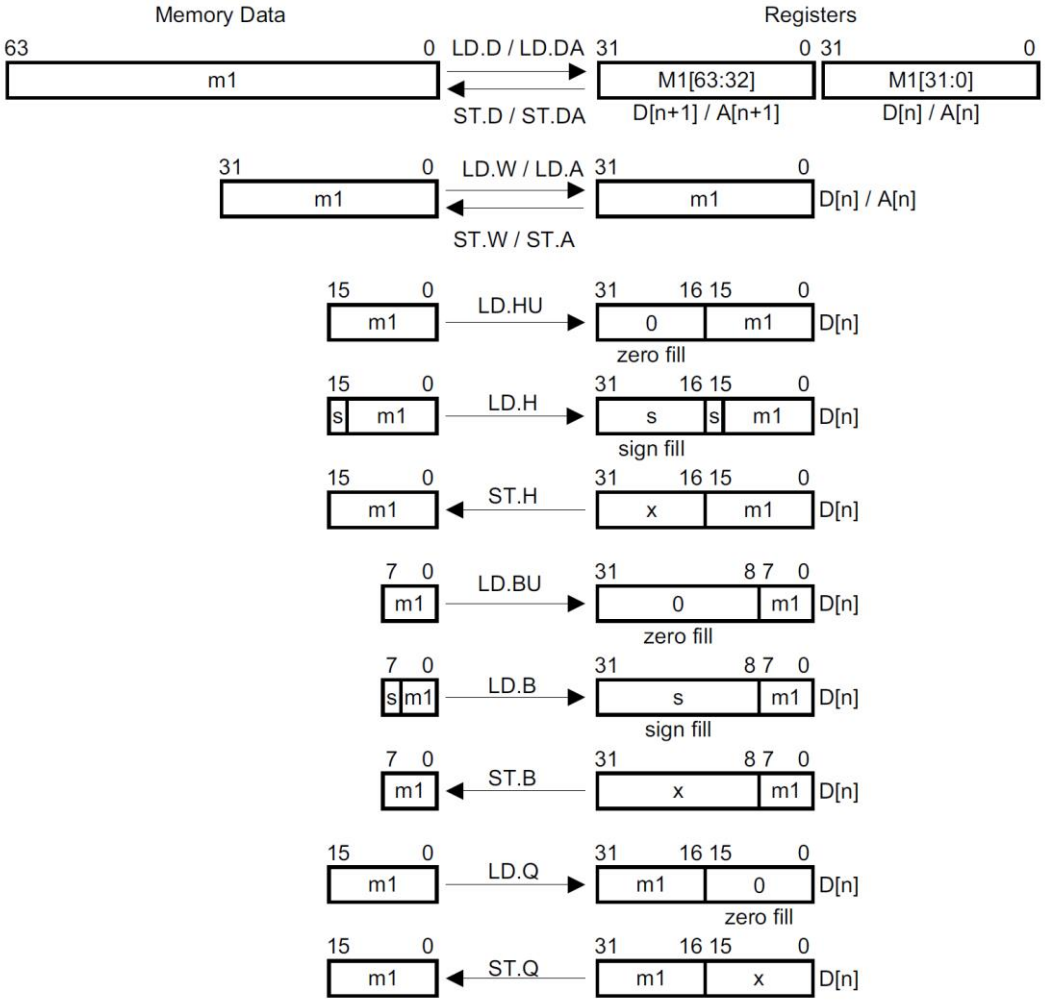
The addressing mode determines the effective byte address for the load or store instruction and any update of the base pointer address register.

Table 2-3 Addressing Modes

Addressing Mode	Syntax	Effective Address	Instruction Format
Absolute	Constant	{offset 18[17:14], 14'b00000000000000, offset 18[13:0]}	ABS
Base + Short Offset	A[n]offset	A[n]+sign_ext(offset10)	BO
Base + Long Offset	A[n]offset	A[n]+sign_ext(offset16)	BOL
Pre-Increment	+A[n]offset	A[n]+sign_ext(offset10)	BO
Post-Increment	A[n+]offset	A[n]	BO
Circular	A[n] / A[n+1+c]	A[n]+A[n+1][15:0] (n is even)	BO
Bit-reverse	A[n] / A[n+r]	A[n]+A[n+1][15:0] (n is even)	BO

(2) 典型指令集

Load and Store Instructions（装载与存储指令）



(2) 典型指令集

Load and Store Instructions（装载与存储指令）

- **LD.A** - Load Word to Address Register
- **LD.B** - Load Byte
- **LD.BU** - Load Byte Unsigned
- **LD.D** - Load Double-word
- **LD.DA** - Load Double-word to Address Register
- **LD.H** - Load Half-word
- **LD.HU** - Load Half-word Unsigned
- **LD.Q** - Load Half-word Signed Fraction
- **LD.W** - Load Word
- **LDLCX** - Load Lower Context
- **LDMST** - Load-Modify-Store
- **LDUCX** - Load Upper Context
- **LEA** - Load Effective Address

- **ST.A** - Store Word from Address Register
- **ST.B** - Store Byte
- **ST.D** - Store Double-word
- **ST.DA** - Store Double-word from Address Registers
- **ST.H** - Store Half-word
- **ST.Q** - Store Half-word Signed Fraction
- **ST.T** - Store Bit
- **ST.W** - Store Word
- **STLCX** - Store Lower Context
- **STUCX** - Store Upper Context

- **IMASK** - Insert Mask
- **INS.T** - Insert Bit
- **INSN.T** - Insert Bit-Not
- **INSERT** - Insert Bit Field

（2）典型指令集

Context Related Instructions（现场保护相关指令）

- SVLCX instruction (Save Lower Context)
- RSLCX instruction (Restore Lower Context)

Save/restore 将context存到CSA/从CSA读取context，不需要指定内存地址

- STLCX instruction (Store Lower Context)
- STUCX instruction (Store Upper Context)
- LDUCX instruction (Load Upper Context)

Store/load 将context存到内存地址/从内存地址读取context，需要指定内存地址

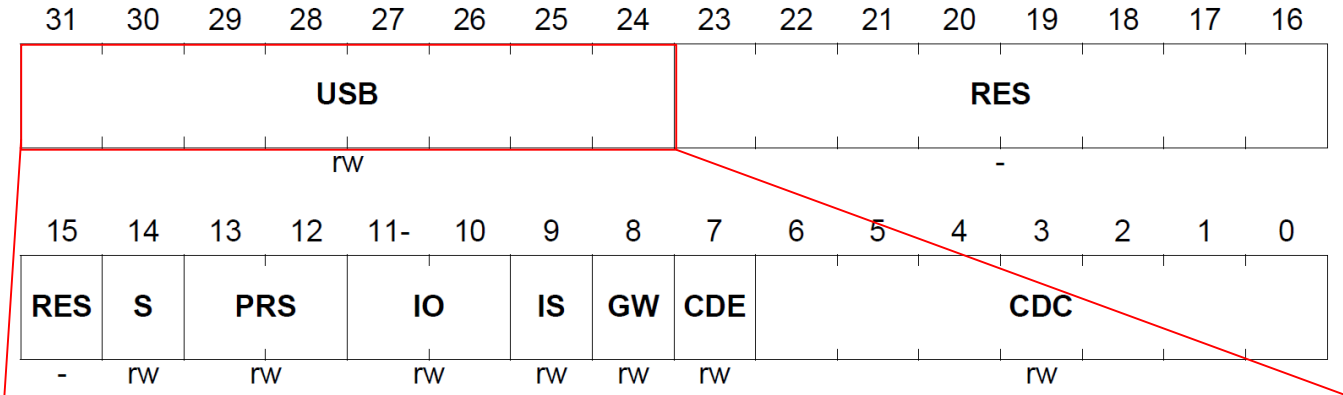
(2) 典型指令集

PSW (Program Status Word) Status Flags and Arithmetic Instructions

Move From Core Register (MFCR)

Move To Core Register (MTCR)

PSW
Program Status Word **($FE04_H$)** **Reset Value: 0000 0B80$_H$**



Field	Bits	Type	Description
C	31	rw	Carry
V	30	rw	Overflow
SV	29	rw	Sticky Overflow
AV	28	rw	Advance Overflow
SAV	27	rw	Sticky Advance Overflow
RES	[26:24]	-	Reserved Field

PSW User Status Bits

（2）典型指令集

System Instructions （系统指令）

- SYSCALL generates a system call trap
- No-Operation (NOP)
- DYSNC and ISYNC
- MFCR and MTCR
- ENABLE and DISABLE
- RET and RFE
- Trap Instructions
- No-Operation (NOP)

Float Point Co-Processor（浮点协处理器指令）

ADD.F
Add Float

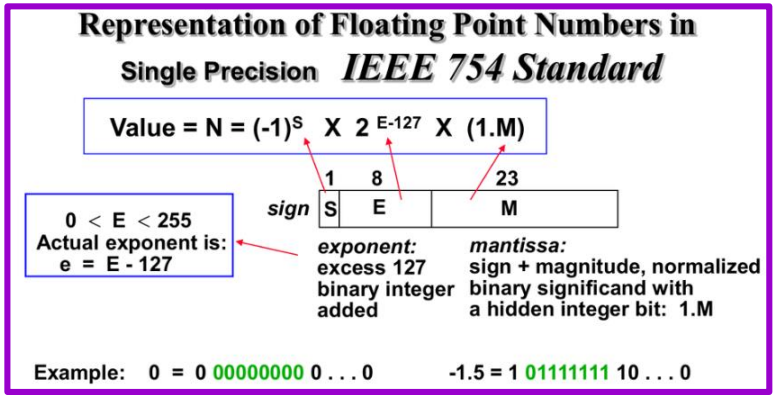
Description

Add the contents of data register D[a] to the contents of data register D[d]. Put the result in data register D[c]. The operands and result are single precision IEEE-754-1985 floating-point numbers. If either operand is a NaN (quiet or signalling), then the return result will be the quiet NaN $7FC00000_H$.

ADD.FD[c], D[d], D[a] (RRR)

31–28	27–24	23–20	19–18	17–16	15–12	11–8	7–0
c	d	02_H	-	1_H	-	a	$6B_H$

```
arg_a = denorm_to_zero(f_real(D[a]);
arg_b = denorm_to_zero(f_real(D[d]);
if(is_nan(D[a]) OR is_nan(D[d])) then result = QUIET_NAN;
else if(is_pos_inf(D[a]) AND is_neg_inf(D[d])) then result = ADD_NAN;
else if(is_neg_inf(D[a]) AND is_pos_inf(D[d])) then result = ADD_NAN;
else {
precise_result = add(arg_a,arg_b);
normal_result = denorm_to_zero(precise_result);
rounded_result = ieee754_round(normal_result, PSW.RM);
result = ieee754_32bit_format(rounded_result);
}
D[c] = result[31:0];
```



（2）典型指令集

Float Point Co-Processor（浮点协处理器指令）

CMP.F
Compare Float

FTOI
Float to Integer

FTOQ31
Float to Fraction

FTOIZ
Float to Integer, Round towards Zero

FTOQ31Z
Float to Fraction, Round towards Zero

MADD.F
Multiply Add Float

SUB.F
Subtract Float

MUL.F
Multiply Float

MSUB.F
Multiply Subtract Float

DIV.F
Divide Float

（3）小结

- 汇编指令代码的基本组成：指令码instruction+操作数operator
- 指令码的基本格式：三个部分，基本指令、指令修饰、操作数修饰，如adds.u
- 汇编指令代码Instruction format、机器操作码Opecode、RTL格式代码，三者之间的关系
- PSW寄存器和汇编指令代码的关系
- 能借助文档读懂常见的指令代码mov、add、sub、mul、div等
- 复习基本逻辑运算AND、OR、NOT、XOR
- 能读懂基本指定代码的组合，如adds, NAND, ORN, SH.AND.T等
- 了解基本系统码：RET、RFE

- 实验中，大量使用C语言编程
- C语言编译器会将其转换为汇编语言，之后转换为机器操作码，最后通过译码电路实现硬件电路操作
- 汇编语言虽然在实验中用到极少，但对于大家了解单片机工作原理，帮助极大

本节结束，谢谢！

Thank you！